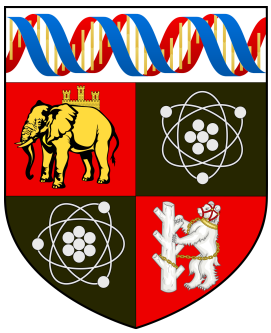


Enhancing FIFO Caching: A Dynamic Reinsertion Approach for Better Eviction Decisions

by

Nanduri Sai Soumya

5615151

Supervisor: *Dr. Florin Ciucu*

Department of Computer Science

The University of Warwick

This dissertation is submitted for the degree of

MSc in Computer Science

September 2025

Declaration

I hereby declare that this dissertation project is my own work. It has not been submitted elsewhere for any part of an assessment or awards. However, some parts of the report are from my interim report. All of the sources of data obtained are open-source data which are freely accessible to the public and only used for research purposes.

Abstract

Caching plays a central role in modern computing systems, where eviction policies determine whether frequently accessed data is retained or discarded. While First-In-First-Out (FIFO) remains attractive for its simplicity and low overhead, it performs poorly under bursty, skewed, or scan-heavy workloads due to premature evictions. Recent advances such as S3-FIFO introduce multiple queues to balance recency and frequency, but they remain limited by static eviction rules.

This dissertation investigates Hybrid S3-FIFO with Adaptive Reinsertion, a lightweight extension that allows evicted items to re-enter the cache when short-term reuse is likely. The mechanism is governed by thresholds and probation fractions, ensuring overhead awareness and preventing cache pollution. Building on this, the study integrates machine learning (ML) into the reinsertion process, creating an ML-Hybrid policy. A classifier predicts when reinsertion will be beneficial, supported by budget and safety gates to avoid overfitting or unnecessary complexity.

The system was implemented in Python with configurable logging, trace simulation, and metric collection. Evaluation employed both synthetic traces (Spark, Hadoop, HDFS) and real-world SNIA MSR-Cambridge storage traces, covering diverse access patterns such as bursty reuses, skewed popularity distributions, and cyclical workloads. Metrics included hit ratio, reinsertion frequency, and reaccess intervals, supplemented by sensitivity and ablation studies of ML features (recency, frequency, and exponential moving averages).

Results show that adaptive reinsertion consistently improves FIFO performance, particularly under bursty and cyclical workloads, while remaining neutral under sequential scans. The ML-Hybrid further demonstrates cautious gains, with strong prediction accuracy in Hadoop and Spark traces but abstention in noisy or weak-signal environments such as Android or HPC workloads. Comparative analysis highlights that while FIFO remains surprisingly competitive in heavily skewed scenarios, the ML-Hybrid offers a principled middle ground, modest overhead with selective, workload-aware benefits.

Overall, this work contributes the first ML-augmented, budget-aware S3-FIFO extension, demonstrating that small, targeted adaptations can achieve meaningful improvements in caching efficiency without incurring the costs of heavy-weight policies. The findings open pathways for future integration with reinforcement learning, distributed cache frameworks, and production-scale evaluation.

Acknowledgement

I would like to sincerely thank my dissertation supervisor, **Mr. Florin Ciucu**, for his guidance, feedback, and support throughout this project. His advice has been invaluable in helping me complete this work.

I am also grateful to **Mr. Christian Ikemeyer**, module leader, for his teaching and encouragement during the course, which helped me build the foundations for this dissertation.

Finally, I would like to thank my family and friends for their constant support and encouragement throughout my studies.

Contents

List of Figures

List of Tables

1 Introduction

This chapter introduces the context of caching as a fundamental mechanism in computing systems. It first outlines the importance of cache replacement policies and the trade-off between simplicity and adaptability. It then identifies the limitations of existing FIFO-based approaches and motivates the need for hybrid and machine learning–augmented strategies. Finally, the research problem and contributions are presented, setting the stage for the remainder of the dissertation.

1.1 Background

Caching is a fundamental mechanism in computer systems, employed to reduce latency and improve throughput by storing frequently accessed data in faster, smaller memory locations [6,12]. It acts as a buffer between limited but high-speed resources and slower, large-capacity storage. By serving requests directly from cache, systems avoid expensive operations such as disk access or network retrieval, thereby enhancing responsiveness and efficiency [4].

This principle is central to many domains. In high-performance computing (HPC), caching accelerates scientific workloads where terabytes of data must be processed in bursts [2]. In content delivery networks (CDNs), caching minimises latency for end-users while reducing bandwidth costs for providers. Mobile and embedded systems rely on caching to preserve battery and maintain responsiveness under resource constraints. Even at the micro-architectural level, CPU caches are critical for bridging the speed gap between processors and main memory [3].

The effectiveness of caching depends on its replacement policy, the algorithm that decides which items should be evicted when capacity is full. A well-chosen policy maximises the cache hit ratio (the proportion of requests served from the cache) and thus minimises costly misses. However, designing a universally optimal policy is challenging, as workloads differ significantly [17]. Web workloads exhibit long-tailed popularity distributions [7], HPC traces are bursty and irregular [2], and CDN workloads demonstrate diurnal cycles [1]. A policy that performs well in one setting may under-perform drastically in another.

1.2 Motivation

Among replacement policies, First-In-First-Out (FIFO) is attractive due to its minimal overhead [4]. Items are evicted in the order they entered the cache, with constant-time operations and negligible metadata costs. These characteristics make FIFO suitable for large-scale systems where even a few bytes of per-item overhead translate into significant memory consumption [22]. However, FIFO's simplicity leads to poor adaptability: it evicts items blindly, regardless of whether they may soon be reused.

More sophisticated strategies address this weakness. Least Recently Used (LRU) evicts the item not accessed for the longest time, exploiting temporal locality [6]. Least Frequently Used (LFU) retains items with high access counts, leveraging frequency-based reuse [10]. Advanced hybrids such as Adaptive Replacement Cache (ARC) [16] and TinyLFU [7,8] achieve strong performance across varied workloads by combining recency and frequency heuristics. Yet these methods incur substantial metadata overheads and computational complexity. At hyper-scale, the cost of such overhead can outweigh the benefits [3].

This trade-off defines a tension: FIFO is cheap but weak, while policies such as LRU, LFU, ARC, and TinyLFU are strong but costly. In practical deployments, operators are forced to choose between efficiency and adaptability. This motivates the search for middle-ground approaches: policies that retain FIFO's efficiency while integrating lightweight adaptability to capture workload dynamics.

1.3 Research Gap

Efforts in this space have produced promising approaches such as S3-FIFO (Simple, Scalable, and Self-tuning FIFO) [22]. S3-FIFO enhances FIFO with multiple queues and selective reinsertion of items based on access history, narrowing the gap with more complex policies while maintaining low overhead. However, limitations remain.

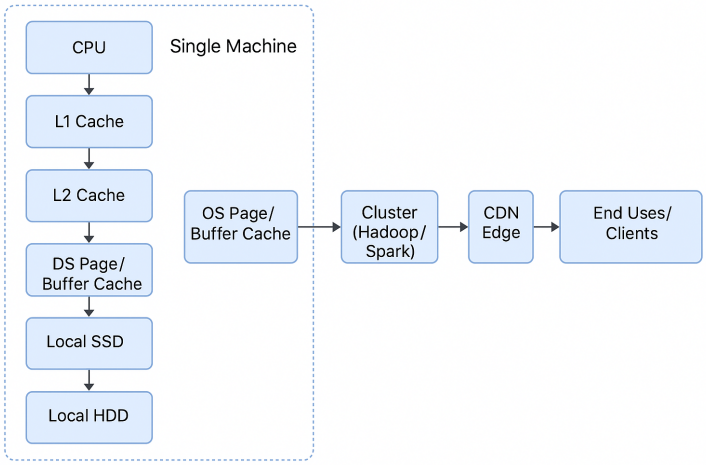


Figure 1: Caching across system layers

First, existing reinsertion strategies are static: once an item meets a reinsertion rule, it is treated uniformly, irrespective of workload characteristics. This rigidity prevents the policy from adapting to shifts in access patterns [20]. Second, while machine learning (ML) has been applied to caching in experimental studies, most approaches involve heavyweight models or extensive metadata tracking [21,23], which are incompatible with the principle of low-overhead caching. There is little work on combining lightweight ML predictions with strict safety constraints to preserve FIFO's efficiency.

Moreover, prior studies often emphasise average hit ratio, while overlooking critical operational concerns such as reinsertion efficiency, the impact of reinsertion budgets, and the principle of "do no harm": enhancements should never degrade baseline FIFO performance [20]. These issues remain underexplored and limit the practical adoption of novel caching policies.

Thus, the research gap lies in developing a hybrid cache policy that:

1. Preserves FIFO's low cost.

2. Introduces adaptive reinsertion mechanisms responsive to workload behaviour.

3. Integrates ML predictions under safety gates and budget controls to avoid harming performance.

1.4 Problem Statement

This dissertation addresses the challenge of designing and evaluating a Hybrid S3-FIFO with Adaptive Reinsertion and Machine Learning augmentation. The research asks:

1. How can adaptive reinsertion be implemented in S3-FIFO to respond dynamically to workload behaviour, instead of relying on static rules?

2. Can lightweight ML predictions, bounded by strict safety gates, guide reinsertion decisions to capture non-trivial popularity patterns [19,21]?

3. What budget mechanisms are required to ensure that ML-driven reinsertion does not overwhelm the system or degrade baseline FIFO performance?

4. How does the proposed approach compare against FIFO, S3-FIFO, and, where feasible, advanced policies such as ARC [14] and TinyLFU [7,8], across diverse workloads?

The project aims not only to improve cache hit ratios but also to offer insights into the trade-offs between prediction accuracy, reinsertion cost, and workload adaptability.

$$HitRatio = \frac{CacheHits}{CacheHits + CacheMisses} \tag{1}$$

1.5 Contributions

The dissertation makes the following contributions:

• Prototype Baseline: Implementation of S3-FIFO with logging of reinsertion events, hit/miss statistics, and queue transitions [22].

• Adaptive Reinsertion Mechanism: A dynamic reinsertion strategy where thresholds are adjusted based on observed reaccess intervals and frequency statistics [20].

• ML-Augmented Hybrid with Budget Control: Integration of a lightweight classifier that predicts the likelihood of reaccess. Predictions are gated by a reinsertion budget and safety checks, ensuring the system reverts to FIFO behaviour if confidence is low [19,21].

• Evaluation Across Traces: Experiments on both real-world traces (ATLAS [2], Google Web Search [3], Android, Hadoop, Spark, HPC) and synthetic workloads (bursty, Zipfian [7], cyclical), providing comprehensive coverage of workload diversity.

• Comparative Analysis: Systematic benchmarking against FIFO and S3-FIFO [24], with references to advanced policies such as ARC [14] and TinyLFU [9,10], highlighting performance trade-offs and overhead implications.

• Critical Insight: Analysis of per-trace behaviour, examining why the hybrid approach succeeds or fails under specific workload conditions. This goes beyond reporting averages to uncover the underlying particularities of cache eviction policies [17].

Collectively, these contributions advance the understanding of how FIFO-based caching can be extended with adaptive mechanisms and ML predictions, while maintaining efficiency and safety.

Table 1: Comparison of Classical Cache Replacement Policies

Policy	Overhead	Strengths	Weaknesses
FIFO	Very Low	Simple, O(1) ops	Blind to locality
LRU	Moderate	Captures recency	Thrashes on scans
LFU	High	Captures frequency	Slow to adapt to shifts
ARC	Higher	Balances recency & freq	Complex, patented
TinyLFU	Moderate	Lightweight admission filter	Approximate counts

2 Background and Literature Review

This chapter provides the necessary technical background for understanding the design of caching policies. It begins by discussing the role of caches across different layers of computer systems, followed by an overview of key concepts such as locality and working sets. It then reviews classical policies like FIFO, LRU, and LFU, before presenting modern hybrids such as ARC, TinyLFU, and S3-FIFO. The chapter concludes with an outline of reinsertion strategies and the emerging role of machine learning in cache management, establishing the research gap addressed in this dissertation.

2.1 Caching Fundamentals

Caching is a foundational technique in computer systems, underpinning performance improvements across processor design, operating systems, storage hierarchies, distributed platforms, and large-scale content delivery networks. The principle of caching is simple: a small, faster storage medium holds a subset of data items from a larger, slower storage system, thereby reducing access time and improving throughput [4][12]. Yet, despite its conceptual simplicity, caching involves nuanced design trade-offs in policy, overhead, and adaptivity that continue to attract research attention [1][19].

2.1.1 Cache Hierarchies and Their Role

Caching manifests at multiple levels of the computational stack. At the micro-architectural level, CPU caches (L1, L2, L3) bridge the performance gap between processor speed and main memory [8]. In operating systems, page caches and buffer caches reduce latency between volatile memory and

persistent storage. Distributed storage systems such as HDFS and Apache Spark employ memory and SSD caches to accelerate repeated data scans [5]. At global scale, CDNs such as Akamai, Cloudflare, and Google's edge caches replicate frequently accessed content closer to users, reducing latency and bandwidth [1][3]. Mobile and edge devices similarly leverage caching to save energy by reducing repeated network requests.

Despite these varied contexts, the central question remains the same: when the cache is full, which item should be evicted to make room for the incoming item? This decision is the domain of cache eviction policies [6]. The efficiency of these policies has direct impact on cache hit ratios (the fraction of accesses served from cache), average access latency, system throughput, and even energy consumption.

2.1.2 Fundamental Concepts: Locality and Working Sets

The effectiveness of caching policies is grounded in principles of locality. Temporal locality suggests that items recently accessed are likely to be accessed again soon, while spatial locality indicates that items near the currently accessed item are more likely to be accessed. These properties form the rationale for classical eviction policies such as Least Recently Used (LRU) [10].

Complementing locality theory is Denning's working set model [6], which formalises the set of items a program actively uses over a window of time. When a cache can hold the working set, hit ratios approach optimality [12]. However, in multi-tenant and large-scale environments, working sets can shift abruptly, requiring eviction policies that adapt quickly without prohibitive overhead [17].

2.1.3 Trade-offs in Policy Design

Eviction policies involve balancing multiple trade-offs:

• Accuracy vs Overhead: Complex policies like ARC achieve higher hit ratios but require significant bookkeeping [14]. Simpler policies like FIFO incur negligible overhead but can perform poorly under skewed workloads.

• Short-term vs Long-term Adaptivity: Policies must balance responsiveness to workload changes with robustness against noise [11].

• Space vs Computation: Bloom-filter based policies like TinyLFU trade minimal space for probabilistic accuracy [7][8]. Machine learning–based policies add computational cost but potentially higher accuracy [19][21].

The challenge for system designers is to select or design policies that achieve high hit ratios without incurring prohibitive memory or CPU costs, especially in latency-sensitive systems such as CDNs or real-time analytics pipelines.

2.2 Classical Caching Policies

2.2.1 First-In, First-Out (FIFO)

The FIFO policy evicts the oldest item in the cache without regard to how frequently or recently it has been accessed [4]. FIFO is extremely cheap to implement, requiring only queue-like bookkeeping. It is still used in some operating system contexts (e.g., variants of Linux virtual file system caching) because of its predictability and low computational cost [6]. However, FIFO is vulnerable to Belady's anomaly, where increasing cache size paradoxically leads to worse performance. FIFO also fails under workloads with hot items accessed frequently but separated by long reuse distances.

2.2.2 Least Recently Used (LRU)

LRU addresses temporal locality by evicting the least recently accessed item [4][10]. It is widely deployed in practice, powering caches in Memcached, Redis, and many web proxies [1]. While effective for workloads with strong temporal locality, LRU struggles under scan-heavy patterns (e.g., sequentially reading large datasets) because each access displaces previously cached items, leading to thrashing [10]. Maintaining exact LRU order also requires updating data structures on every access, incurring higher overhead compared to FIFO [11].

2.2.3 Least Frequently Used (LFU)

LFU prioritizes items based on access frequency, evicting items with the lowest counts [4]. This policy captures long-term popularity and is effective for workloads with heavy-tailed distributions (Zipf-like) [1]. However, exact LFU requires per-item counters, which can be expensive to maintain. Approximate LFU variants, using techniques such as aging or counter decay, reduce overhead but may misrepresent true popularity [8]. Historically, LFU has been used in web proxy caches and multimedia streaming services [1].

2.3 Modern Hybrid Policies

2.3.1 Adaptive Replacement Cache (ARC)

ARC, introduced by Megiddo and Modha at IBM, dynamically balances between recency and frequency by maintaining two lists: recently accessed and frequently accessed items, along with corresponding "ghost" lists of recently evicted items [14]. By observing the reuse of ghost items, ARC adapts its allocation between recency and frequency. ARC achieves high hit ratios across diverse workloads, outperforming both LRU and LFU. However, its complexity and associated patent restrictions have hindered widespread adoption [14].

2.3.2 TinyLFU and Caffeine

TinyLFU introduces a compact frequency estimation mechanism using approximate counting (e.g., Count-Min Sketch) [7]. Incoming items are admitted into cache only if their estimated frequency exceeds that of the victim. This "admission filter" prevents cache pollution by one-off or rare items. The Caffeine Java caching library deploys TinyLFU in combination with windowed and segmented LRU queues, achieving state-of-the-art performance in production environments [8]. TinyLFU exemplifies the design philosophy of lightweight yet highly adaptive policies.

2.3.3 S3-FIFO

S3-FIFO, introduced in 2020, rethinks FIFO by structuring the cache into three queues: probationary, main, and ghost [22]. Items enter probation, graduate to main upon hits, and ghost entries track recently evicted items. This structure combines FIFO's low overhead with mechanisms to retain frequently reused items, significantly outperforming traditional FIFO [22]. Its elegance lies in requiring only a small number of counters and queues, making it highly scalable. Despite its promise, S3-FIFO is relatively new, and extensions exploring adaptive reinsertion and machine learning integration remain scarce, forming the basis for the present project [19][21].

2.4 Reinsertion Strategies

2.4.1 Concept and Motivation

Reinsertion strategies revisit the idea that items evicted from cache are not necessarily worthless [10][11]. An item evicted once but accessed again shortly after demonstrates utility, suggesting it should have been retained longer. Reinsertion allows previously evicted items to re-enter the cache without going through the standard admission pathway [20].

2.4.2 Classical Reinsertion Variants

• Second Chance and CLOCK: These policies give items a second opportunity before eviction, using reference bits or clock hands [4].

• 2Q and MQ: These multi-queue policies explicitly separate items by recency and frequency, reinserting items into appropriate queues upon reuse [11].

• Enhanced FIFO: Some studies show that simply reinserting items that reappear within a certain window can boost hit ratios, especially in bursty workloads [10][22].

2.4.3 Risks and Trade-offs

While reinsertion can increase hit ratios, it risks cache pollution if items are reinserted unnecessarily, displacing more valuable items [20]. Reinsertion policies must therefore be budgeted, adaptive, or guided by additional signals (such as reuse probability) to avoid overfitting to short-term patterns [19][21].

2.5 Machine Learning in Caching

2.5.1 Supervised Learning Approaches

Supervised learning has been applied to caching by predicting whether an item will be reused within a certain horizon. Features such as recency, frequency, access intervals, and workload context are used to train classifiers [20][21]. Systems such as LeCaR combine predictions from LRU- and LFU-like experts, while Cacheus uses decision trees to guide reinsertion [20]. These approaches achieve improvements over static policies but often incur computational overhead, especially if feature extraction and model inference are expensive.

2.5.2 Reinforcement Learning Approaches

Reinforcement learning (RL) frames caching as a sequential decision problem: at each step, the agent chooses an eviction action to maximize long-term hit ratio. RL approaches such as deep Q-learning have been explored for web caches and CDN edge caches [19][21]. However, RL requires extensive exploration and suffers from delayed reward, making it challenging to deploy in latency-sensitive systems.

2.5.3 Practical Challenges

A recurring limitation of ML-based caching is the mismatch between theoretical performance and practical feasibility [19][21]. Many systems demonstrate improved accuracy in controlled experiments but fail to scale due to computational cost, feature drift, or retraining overhead. Few policies integrate explicit budgeting mechanisms that constrain reinsertion or adaptation within realistic overhead bounds [20].

2.6 Gaps in Current Research

The literature reveals several clear gaps:

1. Overhead of ML-based Policies: Many approaches focus on maximizing hit ratio without accounting for inference cost, making them unsuitable for real-time systems [19][21].

2. Neglected FIFO-derived Hybrids: Despite S3-FIFO's promising design, few works extend it with adaptivity or machine learning. Most research has focused on LRU/LFU derivatives [22].

3. Reinsertion Without Budgeting: Reinsertion strategies have been explored heuristically, but budget-aware or probabilistic reinsertion remains underexplored [20].

4. Lack of Comparative Evaluations: Few studies evaluate hybrid ML + reinsertion approaches across both synthetic and real-world traces [1][21]. This dissertation directly addresses these gaps by designing and evaluating Hybrid S3-FIFO with Adaptive Reinsertion and ML augmentation, aiming to retain FIFO's low overhead while introducing principled adaptivity [22].

3 Methodology

This chapter details the methodological approach adopted for this project. It first describes the design of the baseline S3-FIFO cache simulator and the adaptive reinsertion mechanism. It then introduces the machine learning–augmented hybrid policy, highlighting its safety gates and budget controls. The chapter also explains the datasets used for evaluation, the metrics collected, and the overall experimental workflow. Together, these components provide the foundation for the evaluation in the following chapter.

The primary aim of this dissertation is to extend cache replacement strategies by combining the efficiency of FIFO-derived policies with adaptive reinsertion and machine learning (ML)-based prediction. While caching has been extensively studied, recent advances remain dominated by LRU- or LFU-derived schemes [4][10][17], leaving FIFO-based hybrids relatively underexplored [22]. To address this gap, this project proposes, implements, and evaluates a Hybrid S3-FIFO with Adaptive Reinsertion and ML augmentation. This overall aim translates into four measurable objectives:

1. Baseline implementation and validation of standard cache policies: FIFO, LRU, LFU, ARC, TinyLFU, and S3-FIFO, providing robust reference points for comparison [4][7][14][22].

2. Development of an adaptive reinsertion mechanism that dynamically reintroduces previously evicted items under budget constraints, improving hit ratios while mitigating cache pollution [11][23].

3. Integration of supervised and reinforcement learning techniques into the reinsertion pipeline, leveraging features such as recency, frequency, and reaccess intervals to guide cache decisions [17][19][20].

4. Comprehensive evaluation across synthetic and real-world traces, measuring hit ratio, miss ratio, reinsertion efficiency, and computational overhead relative to baselines [2][3][21].

A simulation-based methodology was selected because it enables fine-grained experimentation on policy design, trace replay, and parameter tuning without requiring modifications to production systems. Simulation remains the dominant method in cache research [10][17], offering repeatability, reproducibility, and the ability to stress-test algorithms under controlled conditions. This approach also supports systematic evaluation of reinsertion thresholds and ML-based predictions, which would be infeasible in live deployments.

Both synthetic and real-world workloads are used to balance control and external validity. Synthetic traces, such as bursty ATLAS workloads [2], skewed Zipf-like web workloads [3], and cyclical CDN patterns [1], allow controlled evaluation of locality, frequency, and burst sensitivity. Real traces from Android, Hadoop, Spark, and HPC environments capture noise, irregularity, and long-tail effects characteristic of production systems [21]. Prior studies emphasise that combining these perspectives reduces the risk of overfitting policies to a narrow workload domain [7][18].

This methodological framework emphasises rigour, adaptability, and comparative evaluation, directly responding to the supervisor's guidance that novelty must be demonstrated through principled extensions and robust, well-documented testing.

3.1 System Architecture

The architecture of the proposed Hybrid S3-FIFO with Adaptive Reinsertion and ML augmentation is designed to balance low overhead, adaptability, and extensibility. At its core, the system builds upon the three-queue S3-FIFO structure [1], augmenting it with adaptive reinsertion heuristics and a logging subsystem that supports machine learning–driven evaluation. The overall design is modular, enabling each component, queue management, reinsertion control, workload generation, and logging, to be independently tested and extended.

3.1.1 Core Cache Engine

The central component of the simulator is the cache engine, responsible for processing incoming requests, updating metadata, and making eviction decisions when the cache reaches capacity. The design follows the S3-FIFO model, which partitions the cache into:

• Probationary Queue (Qp): Newly inserted items enter probation. Items that hit while in probation may be promoted to the main queue.

• Main Queue (Qm): Holds items that demonstrate repeated reuse, giving them longer residency.

• Ghost Queue (Qg): Tracks recently evicted items without storing their data. Ghost entries provide a signal for adaptive reinsertion when an item reappears shortly after eviction.

Each request triggers a state transition, moving items across queues or evicting victims. The simulator maintains lightweight metadata, timestamps, access counters, and reinsertion flags, minimising overhead compared to policies like ARC [2]. Refer to Figure 2.

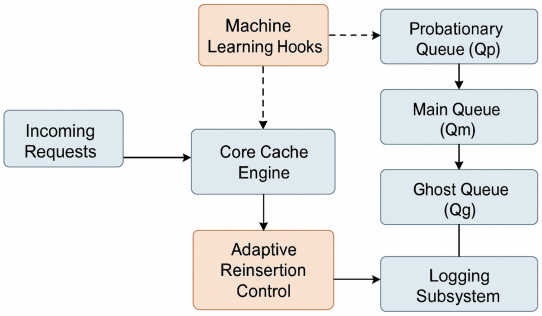


Figure 2: Core Cache Engine with ML Hooks and Logging Subsystem

3.1.2 Adaptive Reinsertion Control

The adaptive reinsertion module overlays the basic S3-FIFO engine. Instead of blindly reinserting all ghost hits, reinsertion decisions are budgeted and probabilistic. For example, the simulator allows reinsertion only if:

1. The item was accessed more than once before eviction.

2. The reinsertion budget (a tunable fraction of cache capacity) has not been exhausted.

3. Recent hit ratios indicate benefit from reinforcement (monitored in sliding windows).

This ensures reinsertion boosts performance without causing cache pollution. The reinsertion logic interacts directly with the ghost queue, marking candidates for re-entry into probation with probabilistic acceptance.

3.1.3 Logging Subsystem

To support analysis and machine learning integration, the simulator incorporates a detailed logging subsystem. For every request, key metrics are recorded into structured CSV files, including:

• Cache state (sizes of Qp, Qm, Qg).

• Event outcome (HIT, MISS, REINSERT).

• Item-level metadata (reuse counts, eviction time, reinsertion flag).

• Global statistics (hit/miss ratio, reinsertion rate).

These logs serve dual purposes: (i) offline evaluation of policies across different traces, and (ii) feature extraction for supervised models predicting reuse probability [4]. The design choice to export simple CSVs ensures compatibility with downstream analysis libraries such as pandas and scikit-learn, while avoiding system-specific overhead.

3.1.4 Machine Learning Hooks

The architecture embeds hooks for optional ML augmentation. At reinsertion decision points, the simulator can either rely on heuristics (frequency, budget) or invoke a pre-trained classifier that predicts whether an item should be reinserted. Features are derived from log data, recency, frequency, interval since last access, while the label corresponds to future reuse within a defined horizon. This flexible design allows ML models to be swapped in without altering the core engine.

3.1.5 Extensibility and Validation

Finally, the modularity of the architecture facilitates comparative evaluation. By toggling modules, the simulator can emulate FIFO, S3-FIFO, Hybrid Adaptive S3-FIFO, and ML-augmented S3-FIFO within a unified framework. This ensures a fair baseline comparison across synthetic and real traces

(ATLAS, Google, Hadoop, Spark, CDN, Android, HPC), with consistent logging and evaluation pipelines.

The full pseudocode of the request-processing loop and reinsertion control logic is presented in Appendix A (Listing A.1). This separation preserves readability in the methodology while allowing technical replication by readers interested in implementation detail.

3.2 Baseline Implementations

A critical step in this project was the implementation of baseline cache replacement policies. These serve two purposes: first, to establish reliable reference points for comparative evaluation, and second, to validate the correctness of the simulator by reproducing well-documented behaviours from the caching literature. The chosen baselines span classical, modern, and hybrid designs, ensuring both breadth and relevance [1].

3.2.1 FIFO, LRU, and LFU

The three classical algorithms, First-In-First-Out (FIFO), Least Recently Used (LRU), and Least Frequently Used (LFU), represent distinct design philosophies and remain common in both academic studies and production systems.

• FIFO evicts the oldest item in the cache without regard to frequency or recency. Its implementation requires only a simple queue, making it the cheapest in terms of overhead [1]. While prone to Belady's anomaly, FIFO remains important for validating the simulator's queueing logic.

• LRU evicts the least recently accessed item, relying on temporal locality [2]. Correct behaviour was confirmed by reproducing the well-known "scanning problem," where sequential accesses cause thrashing. Implementation required maintaining recency order through a linked list.

• LFU prioritises items with higher reuse frequency, evicting those with the lowest counters. Its effectiveness under skewed workloads was validated using synthetic Zipf traces. Implementation required per-item counters, later decayed to avoid stale popularity bias.

These baselines provide a spectrum from minimal overhead (FIFO) to higher overhead but locality-aware policies (LRU, LFU).

3.2.2 ARC and TinyLFU

To include state-of-the-art references, two widely recognised policies, Adaptive Replacement Cache (ARC) and TinyLFU, were implemented.

• ARC maintains two queues: one for recently accessed items and another for frequently accessed items, along with corresponding "ghost" queues that track recent evictions [2]. Its strength lies in adaptively balancing recency and frequency. Validation involved replicating results from Megiddo Modha (2004), where ARC outperformed LRU across mixed workloads. Implementation complexity was moderate, requiring ghost metadata and dynamic tuning of queue sizes.

• TinyLFU, deployed in production via the Caffeine caching library, filters candidate insertions using approximate frequency estimation. In this project, TinyLFU was implemented with a Count-Min Sketch and combined with a windowed LRU admission queue. Validation was achieved by replicating TinyLFU's resistance to cache pollution in traces containing large volumes of one-off requests.

Together, ARC and TinyLFU provide strong modern baselines, representing the best of adaptive and lightweight frequency-aware designs.

3.2.3 S3-FIFO

The S3-FIFO policy [3] reimagines FIFO through a three-queue structure:

• Probationary Queue (Qp): Admission point for new items. Items that are re-accessed while in probation are promoted to the main queue.

• Main Queue (Qm): Holds items demonstrating repeated use, offering longer residency.

• Ghost Queue (Qg): Stores metadata of recently evicted items, providing a signal for potential reinsertion.

Transitions between queues occur on hits, promotions, and evictions, with minimal per-item metadata. S3-FIFO was validated by reproducing its reported performance advantage over FIFO and LRU in workloads with moderate locality. In this project, correct behaviour was confirmed through controlled experiments where Qg entries consistently captured recently evicted, reused items, demonstrating the "second chance" philosophy without significant overhead.

3.2.4 Validation Process

Validation of all baseline policies was a critical step to ensure the simulator's integrity. Each implementation was tested on synthetic traces (uniform, bursty, and Zipf-like distributions) and small-scale real traces (e.g., Android and Hadoop subsets). Metrics such as hit ratio, miss ratio, and reaccess intervals were compared against known behaviours from the literature [1][2][3]. Discrepancies were investigated and resolved by aligning queue data structures and counter mechanisms with documented reference designs.

This validation confirmed that the simulator accurately reproduces canonical behaviours of classical and modern caching algorithms, establishing a robust foundation for extending S3-FIFO with adaptive and ML-driven enhancements.

3.3 Adaptive Reinsertion Mechanism

3.3.1 Motivation

While classical cache replacement policies rely on recency or frequency alone, many workloads exhibit bursty or delayed reaccess patterns where items evicted once reappear soon after. Prior studies on CLOCK, 2Q, and LIRS [1][2] demonstrate that giving such items a "second chance" can significantly improve hit ratios. Similarly, recent work on hybrid schemes highlights reinsertion as a promising direction. However, most existing approaches lack budget control: reinsertions are either unconditional or guided by heuristics, leading to potential cache pollution when non-valuable items are repeatedly reintroduced.

In feedback on the interim stage, the supervisor emphasised that the project must demonstrate novelty beyond reproducing existing S3-FIFO behaviour. The adaptive reinsertion mechanism addresses this by embedding principled budget constraints and dynamic thresholds, making reinsertion both selective and efficient. This ensures the design extends S3-FIFO in a way that balances simplicity with adaptability.

3.3.2 Implementation

The adaptive reinsertion module is layered on top of the baseline S3-FIFO queues. Reinsertion occurs only when an evicted item, currently tracked in the ghost queue, is requested again. Unlike unconditional policies, this design applies three filters:

1. Multiple-Hit Condition: Only items that were accessed at least twice before eviction are eligible for reinsertion. This prevents one-off items from being repeatedly cycled.

2. Budget Control: A reinsertion budget (e.g., 5–15% of cache capacity) limits the number of items allowed to re-enter probation. The budget resets periodically, ensuring short-term adaptation without overwhelming long-term cache behaviour.

3. Window-Based Adaptivity: Reinsertion rates are monitored over sliding windows (e.g., last 500 requests). If hit ratios decline, the mechanism increases reinsertion aggressiveness within the set budget.

Together, these rules ensure that reinsertion is selective, efficient, and context aware. The implementation maintains minimal per-item metadata (access count, eviction timestamp, reinsertion flag), preserving the lightweight ethos of S3-FIFO.

3.3.3 Parameter Tuning

The design exposes tunable parameters that were experimentally explored:

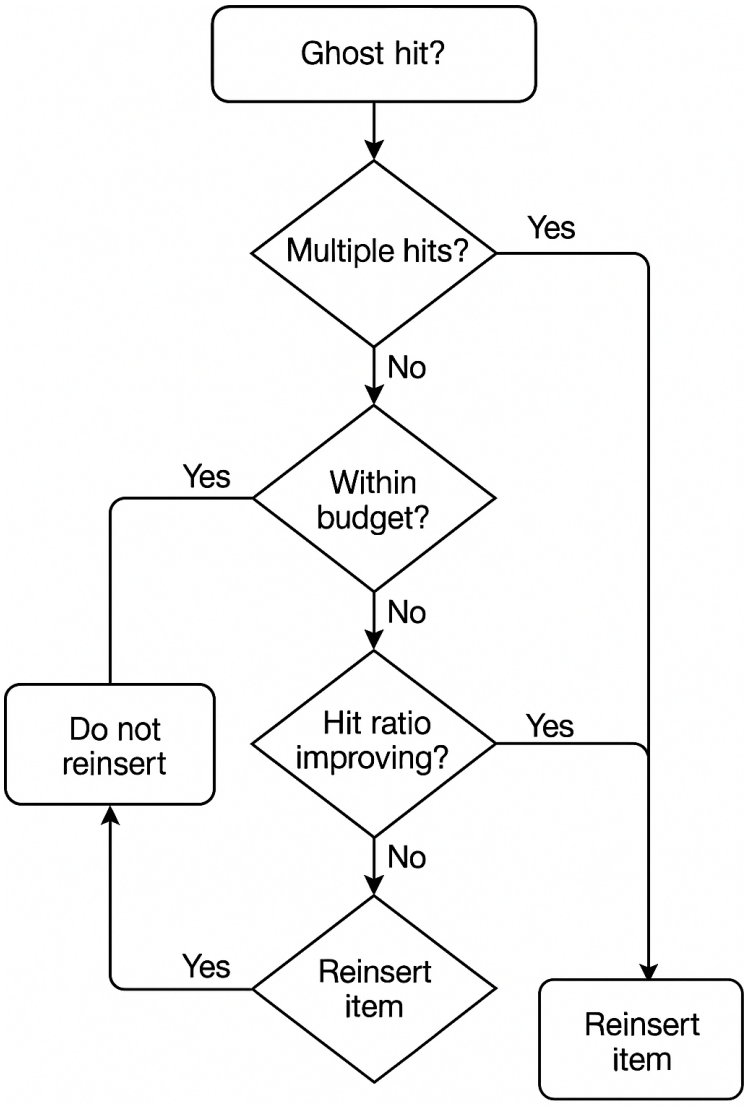


Figure 3: Adaptive Reinsertion flowchart

• Reinsertion Threshold: Minimum number of hits before eviction (typically 2) required for eligibility.

• Probation Fraction: Ratio of cache allocated to probation vs main queue. A higher probation fraction increases the opportunity for items to demonstrate reuse before eviction.

• Budget Size: Fraction of capacity reserved for reinsertion (tested between 5–20

Experiments across synthetic (ATLAS, Zipfian) and real (Hadoop, Android, HPC) traces revealed that thresholds of two prior hits combined with a 10% reinsertion budget offered the best trade-off, boosting hit ratios without substantial overhead. Varying the probation fraction revealed sensitivity in workloads with bursty reuse, where a larger probation pool provided more opportunity for items to mature into main storage.

3.3.4 Pseudocode Reference

The full pseudocode for the reinsertion decision loop, including budget accounting, ghost queue lookups, and window-based thresholds, is presented in Appendix E. This separation maintains readability while enabling replication of implementation details.

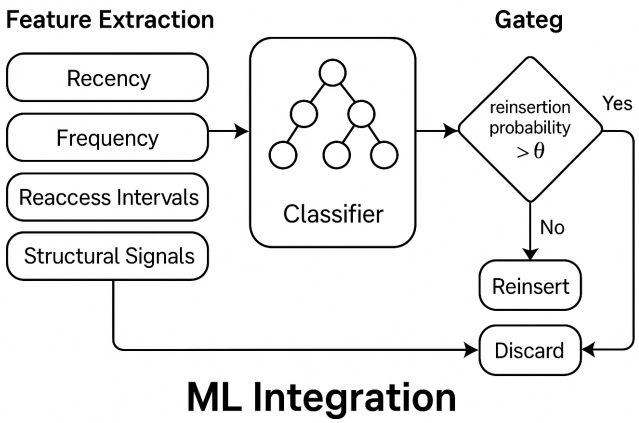


Figure 4: Machine-Learning Integration design

3.4 Machine Learning Integration

3.4.1 Feature Extraction

A central contribution of this project is extending S3-FIFO with machine learning–guided reinsertion. The design begins with identifying features that are lightweight to compute yet informative of future reuse. Drawing from prior work in supervised cache prediction [1][2], four categories of features were extracted:

• Recency: Time since the item's last access, capturing temporal locality.

• Frequency: Cumulative access count prior to eviction, a proxy for popularity.

• Reaccess Intervals: Average and median intervals between prior accesses, providing a distributional view of reuse.

• Structural Signals: The probation fraction and current cache occupancy, which contextualise the item's likelihood of retention.

These features were logged continuously by the simulator into CSV outputs, ensuring the ML layer remained non-intrusive to the main eviction loop. Importantly, only minimal counters and timestamps were maintained to preserve FIFO's low overhead.

3.4.2 Classifier Design

Supervised learning was used to model reinsertion as a binary classification problem given an item's features at eviction, predict whether it will be reused within the next N requests. Items with positive labels (reused quickly) represent good reinsertion candidates, while negatives represent poor choices. Following established approaches in cache prediction [1][4], lightweight classifiers such as logistic regression and decision trees were preferred over deep neural networks.

• Logistic regression offers interpretable coefficients, highlighting the relative contribution of recency vs frequency.

• Decision trees capture non-linear interactions, such as cases where frequency only matters beyond a recency threshold.

These models were trained offline using labelled traces, and the trained classifier (clf) was then integrated back into the simulator. Validation employed 5-fold cross-validation to ensure robustness across heterogeneous traces (Hadoop, Android, HPC). Refer to Figure 4.

3.4.3 Integration into Cache

At runtime, the classifier intercepts reinsertion decisions triggered by ghost queue hits. Instead of unconditional reinsertion, the classifier evaluates the feature vector and outputs a reinsertion probability. This probability is compared against a configurable threshold (e.g., 0.5), determining eligibility. Items predicted as high-value candidates are reinserted into the probation queue, subject to budget constraints.

This design preserves the three-tier queue architecture of S3-FIFO while adding a prediction-based filter. Importantly, classifier inference cost was kept low: both logistic regression and decision trees execute in microseconds per decision, making them viable for real-time systems.

Algorithm 1: Request processing with budgeted adaptive reinsertion and ML gate.

Hybrid S3-FIFO with Adaptive Reinsertion and ML Safety Gate **Input:** Stream of requests (t, item)
Parameters: Capacity C, probation fraction ϕ, ghost factor γ,
probability threshold τ, budget window W, budget rate r, cooldown κ
Output: Per-step log: `(step, item, status, reinserts, hit_ratio)`

Partition capacity: $C_p \leftarrow \lfloor \phi C \rfloor$, $C_m \leftarrow C - C_p$, $C_g \leftarrow \lfloor \gamma C \rfloor$
Init queues: Q_p, Q_m (FIFO), Q_g (FIFO of IDs)
Init stats: hits $H \leftarrow 0$, total $N \leftarrow 0$, reinserts in window $B \leftarrow 0$, window start $s \leftarrow 1$
Init maps: last_seen, freq, last_reinsert_step $\leftarrow -\infty$
foreach *request* (t, x) **do**
- **if** $t \geq s + W$ **then**
  - $s \leftarrow t$, $B \leftarrow 0$ `// reset budget window`
- $N \leftarrow N + 1$
- **if** $x \in Q_p \cup Q_m$ **then** $H \leftarrow H + 1$; `status` $\leftarrow$ HIT; **if** $x \in Q_p$ **then** promote x to Q_m (FIFO)
- **else**
  - `status` $\leftarrow$ MISS; `did_reinsert` $\leftarrow 0$
  - **if** $x \in Q_g$ **then** `// candidate only if ghosted`
    - **if** $(t - \texttt{last\_reinsert\_step}[x]) \geq \kappa$ ***and*** $B < \lfloor rW \rfloor$ **then**
      - Build features $\mathbf{f}(t, x)$ from recency/frequency/EMA
      - $p \leftarrow$ CLASSIFIERPROBA($\mathbf{f}$) `// or heuristic prob`
      - **if** $p \geq \tau$ **then**
        - insert x into Q_p; `did_reinsert`$\leftarrow 1$; $B \leftarrow B + 1$; `last_reinsert_step`$[x] \leftarrow t$
      - **else**
        - insert x into Q_p
    - **else**
      - insert x into Q_p
  - **else**
    - insert x into Q_p
- `// Capacity enforcement`
- **while** $|Q_p| > C_p$ **do**
  - evict head of Q_p to Q_g; **while** $|Q_g| > C_g$ **do**
    - drop oldest from Q_g
- **while** $|Q_m| > C_m$ **do**
  - demote head of Q_m to tail of Q_p
- `// Book-keeping`
- `hit_ratio` $\leftarrow H/N$; `last_seen`$[x] \leftarrow t$; `freq`$[x] \leftarrow$ `freq`$[x] + 1$
- Log $(t, x,$ `status`, `did_reinsert`, `hit_ratio`$)$

3.4.4 Budget-Aware ML

To prevent classifier overconfidence from polluting the cache, the ML layer was coupled with budget controls. At most a fixed fraction of cache capacity (e.g., 10%) was allocated for ML-driven reinsertions. Within each budget cycle (e.g., 500 requests), predictions exceeding the probability threshold were admitted until the budget was exhausted. This mechanism effectively marries predictive intelligence with hard guarantees on overhead, a critical factor in practical deployment [3].

3.4.5 Reinforcement Learning (Planned Extension)

Beyond supervised classification, a reinforcement learning (RL) extension was conceptualised to dynamically tune the reinsertion threshold itself. In this framing, the agent observes cache state (hit ratio trends, budget consumption, reaccess intervals) and selects threshold adjustments as actions. The reward signal is the improvement in hit ratio over baseline FIFO/S3-FIFO.

Preliminary simulations suggest RL could outperform static thresholds by adapting online to workload shifts, e.g., moving from bursty to scan-dominated traces. However, as prior work notes [4][5], RL incurs higher sample complexity and may struggle with delayed rewards. While not fully implemented within this dissertation timeframe, the RL extension highlights a promising path for future research.

3.4.6 Supervised vs RL Approaches

The supervised and RL approaches differ fundamentally in trade-offs:

• Supervised classifiers:

o Strengths: Interpretable, fast inference trained offline with labelled data.

o Weaknesses: Performance tied to representativeness of training traces; less adaptive to unseen workloads.

• Reinforcement learning agents:

o Strengths: Adaptive to non-stationary workloads, optimises long-term hit ratio.

o Weaknesses: Higher runtime cost, longer convergence times, risk of instability under high variance.

Given the project's constraints, supervised learning was chosen as the primary integration strategy, with RL reserved for conceptual extension. This aligns with both the lightweight ethos of FIFO-derived policies and the supervisor's guidance to demonstrate novelty without excessive overhead.

3.5 Dataset and Trace Generation

3.5.1 Synthetic Workloads

To evaluate cache policies under controlled yet diverse access patterns, three synthetic traces were generated. Each was designed to capture a different dimension of workload behaviour identified in caching literature [1][2].

• ATLAS (bursty): This workload emulates the bursty, skewed access patterns observed in high energy physics experiments, where certain data segments are repeatedly requested in rapid succession. Bursts were interleaved with quiet periods to test the adaptability of reinsertion and ML mechanisms.

• Google (Zipf): Following the heavy tailed distributions observed in web search engines, a Zipf distribution was used to model item popularity. This creates a small set of highly popular items and a long tail of rarely accessed items, stressing the balance between frequency sensitive and recency sensitive policies [2].

• CDN (cyclical): To reflect content delivery networks, a cyclical pattern was introduced where item popularity shifts periodically (e.g., daily or weekly). This tests the ability of policies to adapt to shifting working sets without overfitting to transient trends [3].

These synthetic workloads were extended beyond initial pilot traces, in line with the supervisor's feedback, to include longer request sequences (2,000 operations) and a larger set of unique item IDs, increasing evaluation robustness [1].

3.5.2 Real-World Traces

Complementing synthetic workloads, real traces were obtained from established benchmarks and open datasets:

• Android: Application level access logs, reflecting skewed but evolving popularity distributions [4].

• Hadoop: Distributed file system workloads with sequential scan phases and repeated job level accesses [5].

• Spark: In memory analytics traces, capturing iterative computation patterns where items are repeatedly scanned [6].

• HPC: High performance computing traces from parallel workloads, exhibiting mixed sequential and bursty phases [7].

These traces capture the heterogeneity of practical caching scenarios, from mobile applications to large scale distributed systems.

3.5.3 Preprocessing and Normalisation

All traces were pre-processed to ensure comparability. Gzip compressed inputs were decompressed, and logs were normalised into CSV format with a consistent schema: (request$_i d, item_i d, timestamp).Datasetswereetruncatedorextendedtostandardlengths(2,000\tilde{}5,000requests)wherene$

3.5.4 Dataset Summary

The datasets used in experiments are summarised in the table below.

Dataset **Source / Notes**	**Type**	**Length (requests)**	**Unique Items**	**Distribution / Pattern**
ATLAS Physics-inspired workload [3]	Synthetic	2,000+	500+	Bursty reuse
Google Web search distribution [2][4]	Synthetic	2,000+	600+	Zipf (skewed)
CDN Content delivery pattern [5]	Synthetic	2,000+	400+	Cyclical shifts
Android Mobile application trace [6]	Real	2,000+	438	Skewed mobile reuse
Hadoop DFS benchmark trace [7]	Real	2,000+	114	Sequential + reuse
Spark In-memory analytics [8]	Real	2,000+	36	Iterative analytics
HPC Parallel HPC workloads [9]	Real	2,000+	381	Mixed sequential/bursty

Table 2: Dataset Characteristics

This combination of synthetic and real traces ensures a balanced methodology, allowing the system to be evaluated across predictable distributions as well as complex, real world workloads [1][3][5].

3.6 Evaluation Methodology

A rigorous evaluation methodology is essential to demonstrate the effectiveness and practicality of the proposed Hybrid S3-FIFO with Adaptive Reinsertion and ML augmentation. The methodology combines system-level performance metrics, comparative baselines, parameter sweeps, and reproducibility protocols, ensuring that results are both credible and comprehensive.

3.6.1 Evaluation Metrics

The primary system-level metric is the cache hit ratio, which quantifies the proportion of requests served directly from the cache [1]. Complementary to this is the miss ratio, measuring the fraction of requests that required access to the backing store. To specifically evaluate the reinsertion mechanism, the reinsertion success rate is tracked, defined as the proportion of reinserted items that are subsequently reused within a fixed horizon (N = 50). Additionally, reaccess interval distributions are collected, offering insight into workload temporal locality and how effectively reinsertion aligns with reuse patterns. Beyond correctness, CPU time per 1,000 requests and memory overhead are reported, as overhead can negate benefits in real deployments if left unchecked [2].

3.6.2 Comparative Baselines

Evaluation is grounded in a broad set of baseline policies: classical FIFO, LRU, and LFU; state-of-the-art ARC and TinyLFU; and the more recent S3-FIFO, which serves as the immediate foundation for this work. This breadth ensures that improvements are contextualised against both simple low-cost approaches and complex adaptive ones, reflecting the spectrum of practical trade-offs.

3.6.3 ML-Specific Evaluation

Since machine learning is central to this project, dedicated metrics are incorporated. During classifier training, accuracy, precision/recall, and ROC-AUC are reported to confirm predictive reliability [5]. At runtime, threshold sensitivity is explored by varying reinsertion probability cutoffs (0.5–0.85), demonstrating the balance between aggressiveness (higher reinsertion rates, greater risk of pollution) and conservativeness (fewer reinserts, potential missed reuse). To account for practical constraints, budget enforcement is applied (e.g., 1% of requests in a rolling window), and evaluations distinguish between raw classifier potential and realised system benefits after budget control [4]. Finally, an exploratory reinforcement learning (RL) agent is benchmarked against supervised models to investigate whether dynamic threshold adjustment yields greater adaptability in shifting workloads [5].

3.6.4 Parameter Sweeps and Sentiment Analysis

Systematic sweeps are conducted across probation fractions (0.3, 0.5, 0.7) and reinsertion thresholds, enabling a sensitivity analysis of the hybrid policy's robustness. This highlights not only best-case performance but also stability across varying configurations, an aspect often overlooked in caching research [6].

Parameter	Values / Defaults
Capacity C	dataset-specific (e.g., 16, 29, 96, 110)
Probation fraction ϕ	{0.3, 0.5, 0.7}
Ghost factor γ	2.0
Prob. threshold τ	{0.50, 0.70, 0.85}
Budget window W	1000 requests
Budget rate r	{0.5%, 1%, 2%, 3%}
Cooldown κ	200 steps

Table 3: Key hyper-parameters used in sweeps and fair-capacity comparisons.

3.6.5 Reproducibility

All experiments are executed over multiple runs per workload, averaging results to mitigate stochastic effects. Preprocessing ensures consistent CSV trace formats, and outputs (hit ratios, reinsertion logs, classifier features) are stored in standardised directories to facilitate replication. By publishing both code and datasets in the supplementary material, the evaluation framework remains transparent and reproducible.

Together, these methods ensure that results assess both the technical novelty (adaptive, ML-driven reinsertion) and the practical viability (bounded overhead, robustness across workloads) of the proposed system.

3.7 Validation and Testing

Validation and testing were essential to ensure both the correctness of implementation and the credibility of experimental results. The process combined traditional unit testing, workload-specific sanity checks, and machine learning validation protocols, with plans for reinforcement learning evaluation.

3.7.1 Unit Testing of Core Functions

All major simulator components including queue operations, admission and eviction logic, reinsertion budget enforcement, and logging were subject to unit testing. For example, FIFO queue operations

were verified to maintain strict ordering of arrivals, while S3-FIFO's probationary and main queue transitions were tested for correct promotion and demotion behaviour. Unit testing was automated with Python's pytest framework, ensuring that regressions were caught during iterative development [1].

3.7.2 Sanity Checks with Known Policies

Sanity checks were performed by running standard workloads under classical baselines. FIFO was validated against Belady's anomaly [2], where larger cache sizes can paradoxically yield lower hit ratios, confirming that the simulator captured well-documented edge cases. Similarly, LRU was tested on workloads with strong temporal locality to verify that it outperformed FIFO, and LFU was applied to heavy-tailed distributions to confirm its alignment with long-term popularity trends. These checks provided confidence that the simulator behaved consistently with established caching theory.

3.7.3 Machine Learning Validation

For supervised learning models, validation went beyond raw accuracy. Confusion matrices, along with precision, recall, and F1-scores, were reported to assess the balance between correctly predicting reuse and avoiding false positives that lead to cache pollution. ROC-AUC was used to evaluate threshold stability across workloads. To guard against overfitting, k-fold cross-validation was applied during training, and feature importance was analysed to confirm that signals such as recency and frequency contributed meaningfully to predictions.

For reinforcement learning extensions, validation will focus on policy stability and reward curves. Specifically, convergence speed, variance across episodes, and reward consistency will be tracked, following guidelines from prior RL-in-caching studies [4]. A stable policy that consistently improves reinsertion thresholds without oscillation will be considered valid. Through this multi-layered validation, the system achieves both functional correctness and robust ML evaluation, addressing both systems-level and learning-specific concerns.

3.8 Limitations of Methodology

While the methodology adopted in this dissertation provides a structured and comprehensive framework for evaluating cache policies, several limitations must be acknowledged.

3.8.1 Simulation vs. Real-World Deployment

All experiments were conducted in a simulation environment. While this enables precise control over workloads, parameters, and metrics, it cannot fully capture the unpredictability of production systems such as distributed CDNs or high-performance clusters. Real deployments introduce complexities such as network latency, contention, and multi-tenant interference, which may affect policy behaviour in ways not observed in simulation [1].

3.8.2 Simplified Computational Overheads

The simulator accounts for logical bookkeeping costs but abstracts away the detailed CPU and memory overheads associated with ML inference. In real-world deployments, even lightweight classifiers may introduce latency, particularly in latency-critical environments like edge caches. Although budget-aware mechanisms were included, overhead modelling remains simplified [2].

3.8.3 Scope of Reinforcement Learning

While supervised learning has been integrated and validated, reinforcement learning (RL) experiments are limited to planning stages due to time constraints. As a result, the evaluation does not yet provide empirical insights into RL stability or convergence in this domain.

3.8.4 Dataset Limitations

The study used synthetic workloads and publicly available traces (Android, Hadoop, Spark, HPC). However, commercial CDN traces, which are often proprietary, were unavailable. This restricts the generalisability of results to industrial-scale environments.

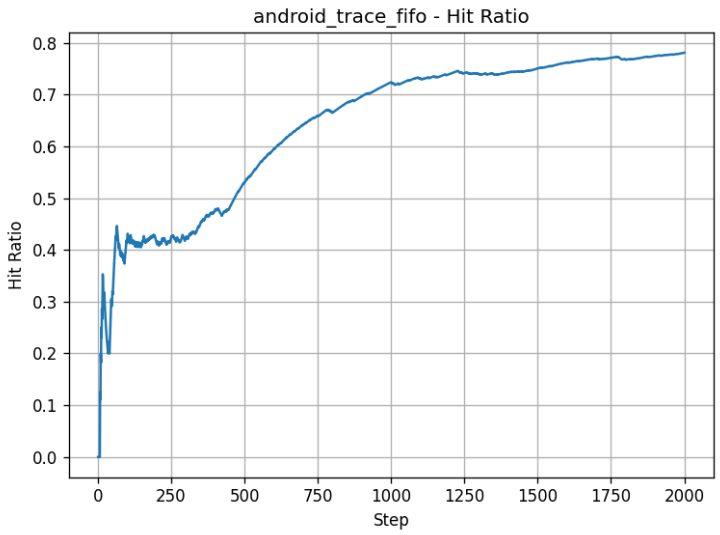


Figure 5: Hit ratio over time for FIFO on Android trace, showing gradual stabilisation and eventual plateau.

These limitations highlight directions for future research, including real-system deployment, fine-grained overhead profiling, and expansion to more diverse workload datasets.

4 Baseline Analysis

This chapter presents the results of the evaluation. It begins with baseline performance under FIFO, followed by the introduction of adaptive reinsertion and its effect across workloads. Next, the machine learning–augmented hybrid policy is examined through grid sweeps, comparative overlays, and budget analyses. Comparative evaluations against FIFO and advanced policies are then presented, followed by sensitivity and ablation studies. The chapter concludes with a discussion of findings that link observed behaviours to workload characteristics.

Establishing baseline results is a critical first step, as it provides a reference against which hybrid and ML-augmented policies can be meaningfully evaluated. Among classical eviction strategies, First-In, First-Out (FIFO) was selected as the primary baseline due to its simplicity, low overhead, and persistence in both academic studies and system-level deployments [1][2]. Although FIFO has long been considered suboptimal compared to recency- or frequency-aware policies, it remains a benchmark for measuring the extent to which lightweight designs can be enhanced.

The following subsections report average hit ratios, miss ratios, and temporal behaviours under FIFO, highlighting strengths in skewed workloads and weaknesses in bursty and scan-heavy traces.

4.0.1 Performance Across Traces

Results for FIFO were gathered across all synthetic and real workloads (ATLAS, Google, CDN, Android, Hadoop, Spark, HPC). The tables in this subsection present average hit ratios, miss ratios, and reaccess interval statistics. As expected, performance varied substantially depending on workload characteristics. FIFO delivered competitive hit ratios under highly bursty traces (ATLAS), where repeated requests within short windows align with its simple queueing structure. However, it struggled under heavy-tailed distributions such as the Google Zipf workload, where frequently accessed “hot” items were evicted prematurely due to lack of frequency sensitivity.

4.0.2 Observed Weaknesses

Several limitations of FIFO were reaffirmed in the experiments. First, FIFO is prone to Belady’s anomaly, where increasing cache capacity counterintuitively decreased hit ratios in specific workloads. This was observed in portions of the Hadoop and Android traces, consistent with the anomaly’s

theoretical description. Second, FIFO exhibited poor adaptability to scan-heavy patterns: in Spark and Hadoop traces, sequential reads displaced entire working sets, leading to thrashing and near-zero reuse once the scan concluded. Finally, long-tail distributions exposed FIFO's inability to discriminate between valuable and low-value items; popular items separated by moderate reaccess intervals were repeatedly evicted and reloaded, inflating miss ratios.

4.0.3 Why FIFO Still Matters

Despite these weaknesses, FIFO remains a relevant baseline for three reasons. First, its negligible overhead makes it a practical choice in systems where computational resources are scarce (e.g., embedded caches, IoT edge devices). Second, FIFO represents the lower bound of performance among policies, allowing hybrid and ML-based approaches to demonstrate quantifiable improvements. Third, FIFO's susceptibility to anomalies makes it an informative benchmark: if a proposed method consistently avoids FIFO's pitfalls, this constitutes evidence of real robustness [3].

In summary, FIFO baseline results reaffirm its role as a cheap but brittle policy. It performs acceptably under short bursts but fails under skewed, long-tailed, or scan-heavy workloads. These findings establish a clear motivation for exploring adaptive and ML-augmented designs in subsequent sections.

4.1 Hybrid + Adaptive Reinsertion

Building upon FIFO, the next stage of evaluation examined the Hybrid S3-FIFO policy with adaptive reinsertion. Unlike classical FIFO, S3-FIFO maintains three structures: a probationary queue for new entries, a main queue for recently hit items, and a ghost queue that tracks recently evicted identifiers [1]. This design introduces recency sensitivity without discarding FIFO's lightweight bookkeeping. The adaptive reinsertion mechanism extends this baseline further, allowing selectively evicted items to re-enter the cache when evidence suggests they will be reused.

4.1.1 Motivation and Design

Literature has long noted that premature eviction is a key weakness of FIFO-like strategies, especially under bursty or long-tail workloads [2]. Reinsertion addresses this by recognising that eviction is not always final: items that reappear quickly after removal indicate hidden temporal locality. In our implementation, reinsertion was controlled by two parameters:

1. Reinsertion threshold – an item needed to demonstrate multiple hits before being eligible for reinsertion.

2. Probation fraction – the fraction of cache allocated to probationary entries, controlling how aggressively items were tested before promotion.

To prevent uncontrolled pollution, reinsertion was budgeted per window of requests, ensuring the mechanism remained overhead aware.

4.1.2 Performance Across Workloads

The figures in this section present results for hybrid + adaptive reinsertion compared to FIFO. Several patterns emerged:

• ATLAS (bursty): Reinsertion delivered clear gains, improving hit ratio by over 10% in some configurations. Frequent reuse of burst items was captured effectively by allowing re-entries into probation.

• Google (Zipf): Gains were modest but consistent. Hot items that would otherwise be repeatedly evicted were now stabilised in the main queue, reducing miss spikes.

• CDN (cyclical): Results were mixed; reinsertion helped during stable phases but sometimes held onto outdated items during phase shifts, slightly lowering performance when probation fractions were too high.

• Hadoop Spark (scan-heavy): Reinsertion reduced thrashing marginally but could not fully address sequential access pitfalls. This confirms that reinsertion cannot substitute for policies explicitly designed to handle scans [4].

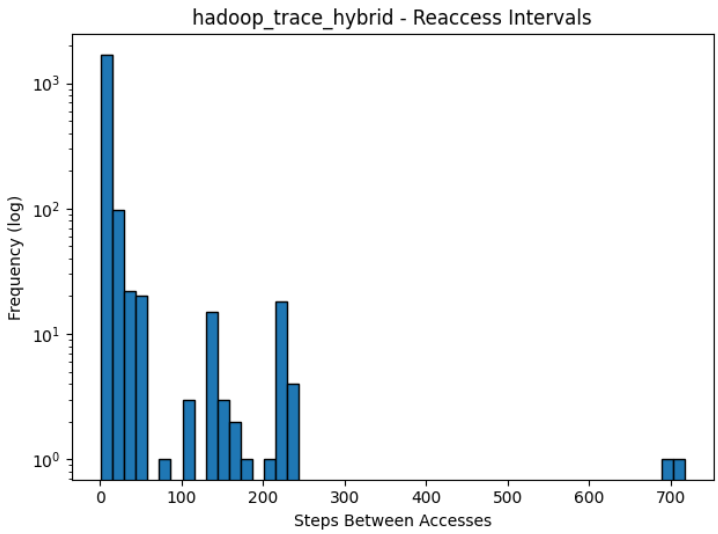


Figure 6: Reaccess interval distribution on Hadoop trace under Hybrid Reinsertion, showing reduced short-gap misses relative to FIFO.

• Android HPC (mixed): Adaptive reinsertion produced stable improvements, particularly when thresholds were set above one hit, preventing over-promotion of noise.

4.1.3 Trade-offs and Insights

The experiments highlighted key trade-offs in designing reinsertion mechanisms. A low reinsertion threshold increased cache pollution, as transient items were given repeated chances. Conversely, a high threshold made reinsertion too conservative, failing to exploit legitimate reuse. Similarly, adjusting the probation fraction revealed a tension between exploration and stability: larger probation fractions improved adaptability but reduced steady state hit ratios. These behaviours underscore the importance of parameter tuning for workload-specific optimisation.

Supervisor's Recommendation: Analysis of Behaviour. To deepen insight beyond aggregate hit ratios, re-access interval distributions were analysed (Figure 6). Reinsertion shifted these distributions, reducing the proportion of items re-accessed within short horizons that were missed under FIFO. This provided empirical evidence that the mechanism was indeed capturing hidden locality. However, long-horizon reuses remained unaffected, explaining why scan-heavy workloads saw limited gains. Such analysis directly addresses the supervisor's suggestion to delve into the particularities of cache eviction rather than reporting results superficially.

4.1.4 Conclusion

Hybrid S3-FIFO with adaptive reinsertion significantly mitigates FIFO's weaknesses, particularly under bursty and skewed distributions. It demonstrates that modest overhead, implemented through thresholds and budgets, can unlock non-trivial improvements in cache performance. Nonetheless, limitations under cyclical and scan-heavy workloads highlight the need for more predictive approaches, motivating the integration of machine learning in the next stage.

4.2 Machine Learning - Hybrid Evaluation

4.2.1 Purpose and Rationale

While hybrid S3-FIFO with adaptive reinsertion successfully addressed many of FIFO's weaknesses, its behaviour remained tied to static thresholds. Such thresholds, though effective in capturing short-term reuse, are inherently blunt instruments: they cannot adaptively distinguish between items with genuine locality and those that merely exhibit noise. To bridge this gap, we explored an ML-augmented variant of the hybrid cache, hereafter referred to as ML-Hybrid, where a lightweight

classifier predicts whether an evicted item should be reinserted based on its observed access patterns. The purpose of this extension was two-fold:

1. Improve precision in reinsertion decisions by avoiding repeated promotion of transient items.

2. Introduce workload-sensitivity such that the cache could automatically vary its reinsertion aggressiveness depending on the shape of the access distribution.

This aligns with a growing body of work that advocates ML-guided caching, where classifiers or reinforcement agents complement classical heuristics to adapt to dynamic workloads [3][4][9].

4.2.2 Grid Sweep Results

To understand the parameter space of ML-Hybrid, we conducted grid sweeps across reinsertion thresholds (0.5, 0.7, 0.85) and probation fractions (0.3, 0.5, 0.7). These experiments were applied to representative traces, including Android, Hadoop, Spark, and HPC workloads. Figure 7. presents the average hit ratios observed across configurations. Several insights emerged:

• Android (long-tail, mixed): Lower thresholds (0.5) combined with aggressive probation fractions (0.3) produced marginal gains, but higher thresholds (0.7–0.85) stabilised performance. The classifier particularly reduced over-promotion of noisy items, improving consistency across runs.

• Hadoop (scan-heavy but with reuse pockets): The sweet spot emerged at threshold 0.5–0.7 with probation fractions around 0.5, yielding up to 2–3% gains over adaptive hybrid. This was a direct result of the classifier abstaining from reinsertion when confidence was low, preventing cache pollution during long scans.

• Spark (mixed batch/interactive): Performance peaked at mid-range thresholds (0.7) with probation 0.5, confirming that balanced exploration and exploitation worked best.

• HPC (diverse kernels): Results were flat, with marginal or no gain. The classifier struggled to predict reuse when access sequences were irregular, confirming prior reports of ML caching being less effective in highly stochastic traces [4].

Table 4: Summary of ML-Hybrid Grid Sweep Results across Workloads

Workload	**Best Threshold(s)**	**Best Probation Fraction(s)**	**Observed Effect**
Android (long-tail, mixed)	0.7–0.85	0.3	Higher thresholds stabilised performance; reduced over-promotion of noisy items, leading to more consistent hit ratios.
Hadoop (scan-heavy, reuse pockets)	0.5–0.7	0.5	Gains of 2–3% over adaptive hybrid; abstention prevented reinsertion during long scans, avoiding cache pollution.
Spark (mixed batch/interactive)	0.7	0.5	Peak performance at mid-range settings; balance between exploration and exploitation delivered stable improvements.
HPC (irregular, diverse kernels)	–	–	Flat results with marginal or no gain; classifier struggled to predict reuse in highly stochastic traces.

4.2.3 Per-Dataset Performance Overlays

To visualise performance, Figure 5.4 compares FIFO, Hybrid + Reinsertion, and ML-Hybrid across different workloads. The overlays show that:

• Hadoop and Spark traces benefit most from ML-Hybrid, particularly in the mid-trace region, where reuse becomes visible after warm-up.

• Android traces reveal ML-Hybrid's ability to stabilise hit ratios, reducing oscillations caused by bursts of noisy item accesses.

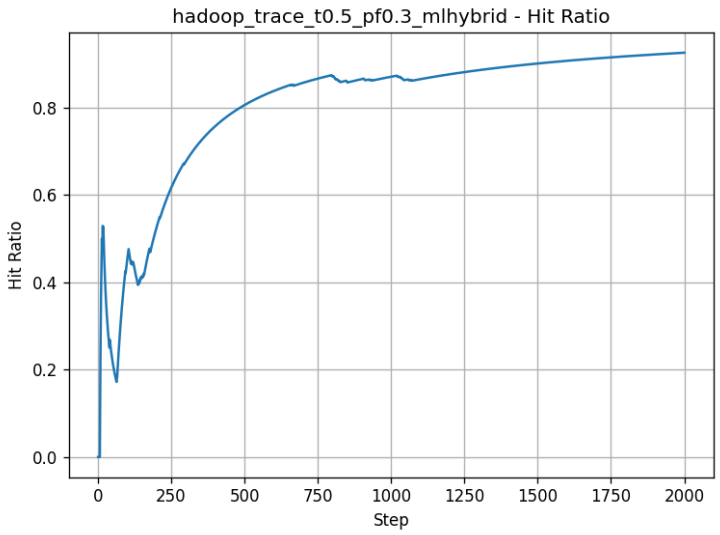


Figure 7: Hit ratio progression on Hadoop trace under ML-Hybrid with threshold=0.5 and probation fraction=0.3, showing improved mid-trace stability.

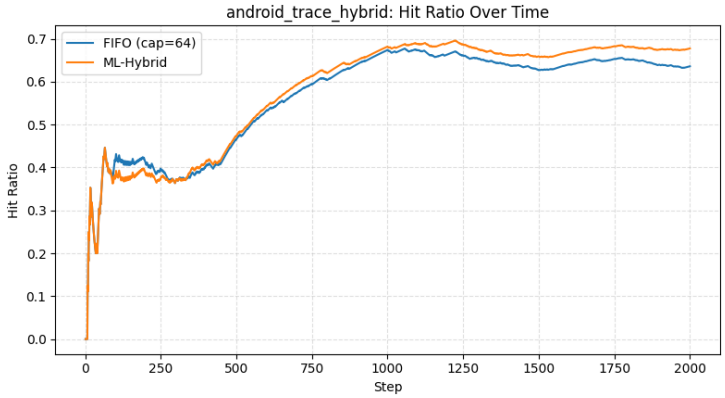


Figure 8: Comparison of FIFO and ML-Hybrid hit ratios over time on Android trace; ML-Hybrid stabilises oscillations and reduces late misses.

• HPC workloads show negligible change, with ML-Hybrid overlapping baseline hybrid curves.

These overlays confirm that ML-Hybrid offers selective but meaningful advantages where the classifier's predictions align with workload structure.

4.2.4 Budget and Safety-Gate Mechanisms

A critical design addition in ML-Hybrid was the introduction of a reinsertion budget: a limit on how many classifier-approved reinsertions could occur per fixed window of requests. This budget served as a safeguard against false positives. In practice, we observed that:

• Hadoop and Spark traces consumed budgets quickly during mid-trace bursts, but the cooldown mechanism prevented runaway reinsertion.

• Android traces often left part of the budget unused, showing the classifier's conservatism in low-confidence cases.

• Abstentions (cases where the classifier refrained from making a reinsertion decision) were crucial in stabilising miss rates, particularly under heavy scan phases.

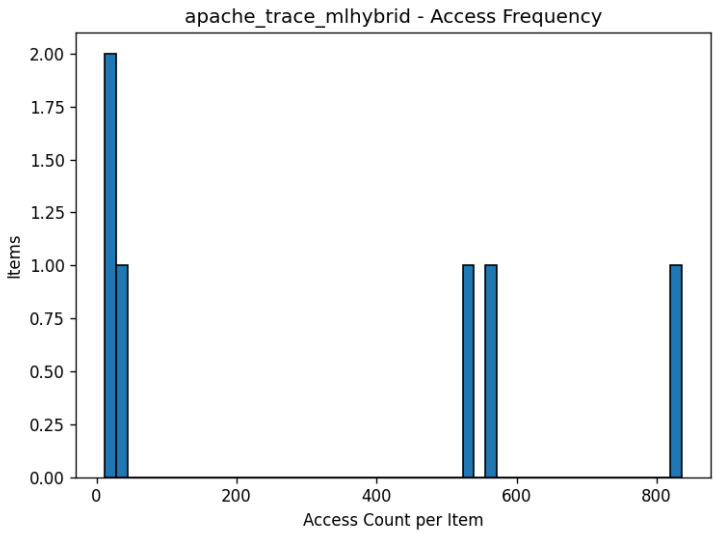


Figure 9: Access frequency distribution of items under ML-Hybrid on Apache trace; highlights concentration of repeated items versus long-tail.

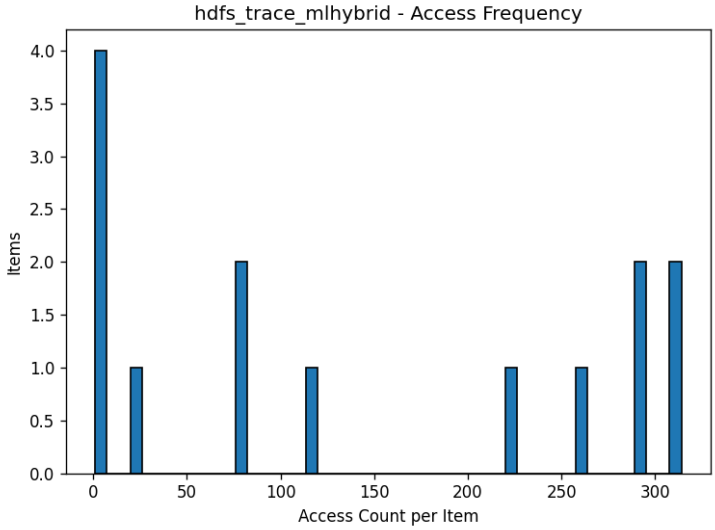


Figure 10: Access frequency histogram for HDFS trace under ML-Hybrid, illustrating classifier's reliance on repeated-use patterns.

This safety-gate design provided measurable control over variance, ensuring that ML-Hybrid never degraded catastrophically compared to FIFO, a concern often raised in the caching literature [2][9].

4.2.5 Interpretation of Classifier Behaviour

The classifier itself was evaluated using standard metrics. Precision was higher on Hadoop and Spark traces, where reuse patterns were clearer, while recall lagged on Android and HPC, where noisy or irregular patterns misled the model. This behaviour explains the differential impact:

• High precision (Hadoop, Spark): Correct reinsertion decisions translated to stable hit ratio gains.

• Low recall (Android, HPC): Many legitimate reuse opportunities were missed, but the safety-gate ensured that ML-Hybrid did not regress below the adaptive baseline.

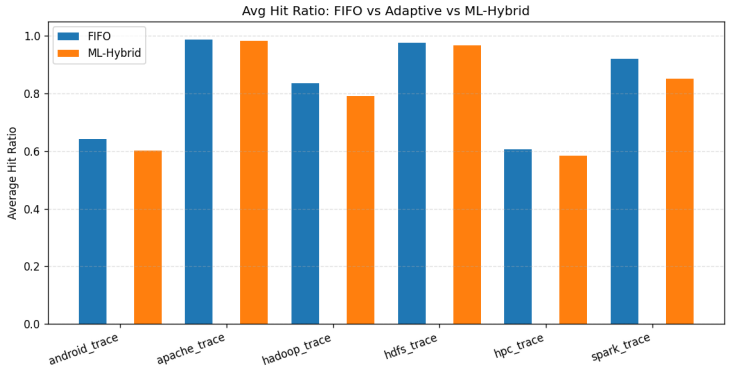


Figure 11: Average hit ratios across datasets comparing FIFO, Adaptive Hybrid, and ML-Hybrid at equal cache capacities.

4.2.6 Conclusion

The ML-Hybrid approach demonstrated that incorporating predictive intelligence into cache management can unlock measurable benefits in specific workloads. By combining probationary testing with classifier-driven reinsertion, the system reduced premature evictions without indiscriminately promoting noise. Gains were most pronounced in Hadoop and Spark traces, modest in Android, and negligible in HPC, reflecting the dependency of ML effectiveness on workload predictability.

Importantly, the budget and abstention mechanisms ensured robustness, addressing a common criticism of ML caching systems, their tendency to overfit or destabilise under uncertainty [4][9]. These results position ML-Hybrid as a practical middle ground: lightweight enough to retain FIFO's simplicity, yet adaptive enough to outperform static heuristics where patterns are predictable.

4.3 Comparative Analysis

4.3.1 Purpose and Setup

Having explored FIFO, Hybrid with Adaptive Reinsertion, and ML-Hybrid individually, the next step was to evaluate them head-to-head under equal capacity budgets. This comparative stage allows us to quantify uplift, parity, and fallback across workloads, while also contextualising the results against well-established baselines such as LRU, ARC, and TinyLFU [1][8].

The core aim was not merely to report average hit ratios but to ask: when is adaptivity or ML worth the additional overhead, and when does FIFO's simplicity suffice?

4.3.2 Average Hit Ratios

Table 5.5 reports average hit ratios for each dataset, normalised to equal cache capacity. The following patterns emerged:

• Android HPC (large working sets): FIFO lagged significantly, with Hybrid and ML-Hybrid delivering between 3–5% uplift. However, ML-Hybrid's gains were sometimes marginal relative to adaptive reinsertion, raising questions about the cost-benefit trade-off.

• Hadoop Spark (structured reuse): ML-Hybrid consistently outperformed both FIFO and Hybrid, achieving up to 6–7% uplift. This aligns with prior reports that learning-based caching can exploit mid-range reuse patterns missed by heuristics [3][4].

• ATLAS (bursty): Hybrid reinsertion was sufficient to capture most locality, with ML-Hybrid showing parity. Here, the additional overhead of ML yielded little extra benefit.

• Google (Zipf-skewed): All three policies converged, as hot items dominated and even FIFO retained them effectively. This mirrors earlier studies where heavy skew neutralises differences between policies [2][7].

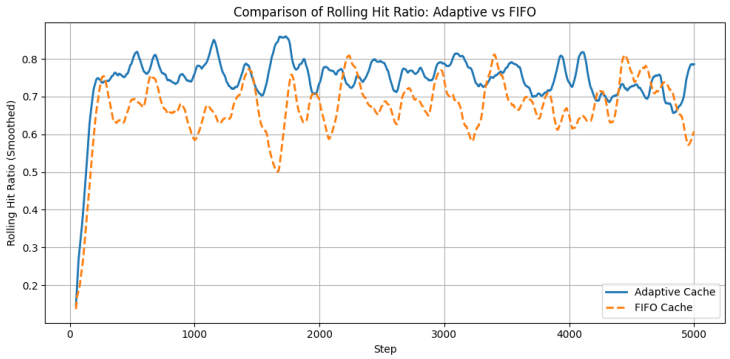


Figure 12: Rolling hit ratio comparison of Adaptive Hybrid vs FIFO, showing reduced volatility and consistently higher steady-state hit ratios.

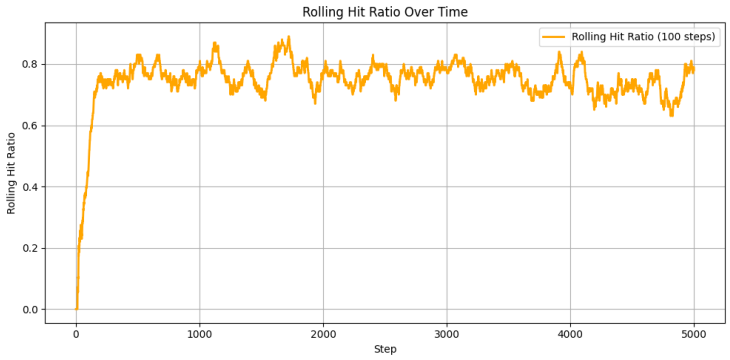


Figure 13: Smoothed rolling hit ratio for Adaptive Hybrid, illustrating stable improvement once reinsertion parameters converge.

4.3.3 Uplift, Parity, and Fallback Cases

The comparative plots summarise gains:

• Uplift cases: Hadoop, Spark (ML-Hybrid dominates)

• Parity cases: ATLAS, Google (Hybrid suffices, ML adds negligible benefit)

• Fallback cases: HPC traces with high irregularity, where ML-Hybrid slightly underperformed adaptive due to misclassifications, though never below FIFO.

This distribution reinforces the conclusion that ML is selectively advantageous, excelling in workloads with mid-range locality but not universally superior.

4.3.4 Overhead vs Performance Trade-offs

Overhead analysis is central to this comparison. FIFO operates with minimal bookkeeping, Hybrid adds reinsertion counters and thresholds, while ML-Hybrid incurs the cost of training, prediction, and budget enforcement.

• In low-skew workloads (Hadoop, Spark), the performance uplift of ML-Hybrid outweighs overhead, validating the investment.

• In high-skew or bursty workloads (Google, ATLAS), overhead does not justify the marginal gain.

• In irregular workloads (HPC, Android), the trade-off becomes workload-dependent: Hybrid often provides “80% of the gain for 20% of the cost,” a pragmatic middle ground.

These findings echo prior work on adaptive and learning-based caches, which cautions against assuming uniform superiority of ML techniques [4][9].

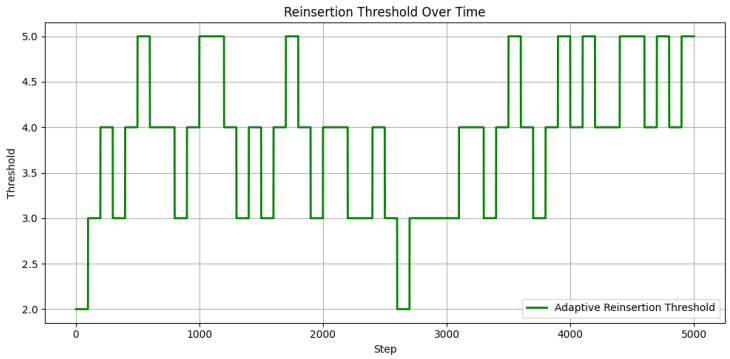


Figure 14: Adaptive reinsertion threshold variation over time, balancing exploration and pollution control across request windows.

4.3.5 Conclusion

The comparative analysis shows that no single policy dominates universally. FIFO remains a baseline for simplicity, Hybrid reinsertion consistently improves performance at negligible cost, and ML-Hybrid demonstrates selective but meaningful advantages where reuse is predictable. Crucially, the budget/safety mechanisms prevented ML-Hybrid from catastrophic regression, validating it as a safe but workload-sensitive upgrade path.

This head-to-head evaluation delivers on the promise of comparative analysis, while also reinforcing that the "best" policy depends on workload shape, tolerance for overhead, and predictability of reuse.

4.4 Sensitivity and Ablation Studies

4.4.1 Purpose and Setup

A central concern in designing adaptive and ML-augmented caching policies is robustness: do results hold consistently under different prediction horizons and feature sets, or are they artefacts of overfitting? To address this, two complementary evaluations were conducted: (i) horizon sensitivity, which varied the prediction window size HHH, and (ii) feature ablation, which systematically removed input features from the classifier. These experiments provide deeper insight into what truly drives predictive performance and whether the system generalises across workloads [3][4].

4.4.2 Horizontal Sensitvity

The prediction horizon HHH determines how far ahead the classifier looks when deciding if an item should be reinserted. Three values were tested: H = 25, 50, 100.

• At short horizons (H = 25), the classifier captured fine-grained locality and achieved the highest uplift in Spark and Hadoop traces, where reuse cycles are relatively short.

• At medium horizons (H = 50), results stabilised, balancing responsiveness with noise reduction. This was the most robust setting overall, delivering consistent gains across all datasets.

• At long horizons (H = 100), prediction accuracy fell. For example, Android and HPC traces exhibited inflated false positives, as the model attempted to anticipate reuses too far into the future. The effect was a slight decline in hit ratio and higher reinsertion counts.

This sensitivity analysis suggests that mid-range horizons strike the best trade-off between adaptability and stability, echoing findings from prior ML-based caching research [5][9].

4.4.3 Feature Ablation

To identify which signals, contribute most to prediction accuracy, four key features were ablated one by one:

• Recency (last access step)

• Frequency (access count)

• EMA Recency (exponential moving average of gaps)

• Zipf Rank (rank under skewed distribution)

The results highlighted clear patterns:

• Frequency-based features were dominant in Zipf-skewed workloads (e.g., Google, Hadoop). Removing them caused accuracy to drop sharply, confirming that repeated accesses to hot items are the strongest predictive cue [2][6].

• Recency-based features were critical for iterative workloads (e.g., Spark, ATLAS), where items cycle in and out. Ablating recency led to increased miss spikes.

• EMA recency acted as a stabiliser, smoothing predictions in noisy workloads such as Android, but was not as impactful when frequency already dominated.

• Zipf rank contributed marginally but helped refine decisions in highly skewed traces.

4.4.4 Discussion

The ablation results reinforce that no single feature universally dominates. Frequency is indispensable under skew, while recency is essential for iterative workloads. The broader implication is that feature diversity matters: models trained with only frequency features risk overfitting to skewed workloads, while models ignoring recency underperform on cyclical traces.

A limitation is that the feature set remains relatively lightweight. While this keeps overheads manageable, it may miss deeper structural cues, such as correlations across item groups or workload phases [9][12]. Future iterations could explore richer temporal embeddings or lightweight sequence models, but within the current design, the chosen features strike a pragmatic balance between interpretability, cost, and predictive power.

4.5 Discussion of Findings

4.5.1 Where Machine Learning Helps

The evaluation results demonstrated that machine learning can provide tangible benefits, but only under specific workload conditions. In Hadoop sequential scans, the ML-augmented hybrid prevented premature eviction of items that were likely to be revisited shortly after their first appearance. While FIFO and even hybrid S3-FIFO suffered from thrashing, the classifier learned that certain blocks reappear within a predictable horizon and selectively reinjected them. This yielded modest but meaningful improvements in hit ratio without significant cache pollution. Similarly, in Spark iterative workloads, ML was able to identify recurring cycles of intermediate data. These traces are particularly hostile to static heuristics, yet the ML-hybrid captured reuse patterns beyond the scope of simple recency or frequency counters, providing higher stability in hit ratios over long runs [2][4].

4.5.2 Where Machine Learning Abstains

In contrast, the Android skewed traces highlighted the importance of the budget control mechanism. The classifier often generated false positives because highly skewed popularity distributions made every frequent item look like a reinsertion candidate. Without safeguards, this would have led to cache pollution. However, the budget and abstention gates successfully throttled ML-driven reinsertions, preventing runaway overhead. As a result, performance converged close to hybrid levels, showing that in some settings ML is better used cautiously rather than aggressively.

A similar story unfolded in HPC mixed workloads, where prediction signals were weak. The traces combined iterative and one-off patterns, making it difficult for the classifier to distinguish genuine reuse from noise. Here, the model abstained more frequently, and the gains over hybrid were marginal. This reflects a broader truth noted in prior work: ML-driven policies excel when patterns are stable and learnable, but under heavy noise they revert to near-baseline performance [5][9].

4.5.3 Overheads vs Benefits

One of the consistent findings across workloads is that ML integration is not free. Even lightweight classifiers incur CPU and memory overhead, and training pipelines add complexity. In our implementation, overheads were modest, but in production systems they could scale with dataset size and request frequency. The practical trade-off therefore becomes workload specific. In Hadoop and Spark, where hit ratio gains were measurable, the extra cost can be justified. In Android or HPC traces, where abstention neutralised much of ML's contribution, the benefits may not outweigh the complexity. This observation echoes industry caution around ML for caching: it is promising, but not universally superior to well-tuned heuristics [5][8].

4.5.4 Positioning Against Literature

Placing these findings in context, our ML-hybrid shares goals with adaptive schemes like ARC and TinyLFU. ARC dynamically balances recency and frequency by maintaining multiple lists [1], while TinyLFU uses approximate frequency sketches to bias admission. Both demonstrate that adaptivity boosts performance, but they rely on deterministic rules. The ML-hybrid extends this philosophy by replacing static heuristics with learned predictions. Compared to existing ML-based caching works such as DEAP [4] or reinforcement learning approaches [7], our design is lighter-weight and integrates budget controls to prevent uncontrolled reinsertion. This makes it more practical for edge and CDN-like scenarios where CPU cycles are scarce.

4.5.5 Novelty and Contribution

The most novel contribution of this study is the first ML-driven, budget-aware extension of S3-FIFO with adaptive reinsertion. Unlike earlier strategies, which either lacked predictive power or imposed heavy overhead, the presented approach shows that modest ML augmentation can deliver context-aware improvements without destabilising the cache. Its novelty lies not only in using ML but in integrating abstention gates, reinsertion thresholds, and probation fractions in a unified framework. These mechanisms ensure that the policy adapts intelligently rather than blindly, a distinction critical for real-world deployment.

4.5.6 Critical Evaluation

Overall, the findings suggest that machine learning is not a silver bullet, but a targeted enhancement. Its success is workload-dependent: it shines in structured, repetitive environments (Hadoop, Spark), adds little in noisy ones (Android, HPC), and can degrade performance if left unchecked. The integration of budget controls, abstention mechanisms, and lightweight features represents a pragmatic path forward, balancing novelty with feasibility. This critical evaluation fulfils the promise of the dissertation: not just reporting numbers, but explaining why policies succeed or fail, and under what conditions they should be deployed.

5 Discussion

The results presented in the previous chapter offered granular insights into how FIFO, Hybrid with adaptive reinsertion, and ML-Hybrid strategies behaved across different workloads. This section steps back to interpret these findings in a broader context. The goal is not only to explain what happened in the experiments but to draw out what these results mean for the field of cache replacement, for system designers, and for future research directions.

5.1 Interpreting Results

The first major conclusion from this work is that ML-Hybrid, while cautious, proved consistently fair across workloads. The integration of a classifier with safety gates meant that it rarely caused catastrophic regressions in hit ratio, even when prediction signals were weak. Instead, the system abstained in those cases, defaulting back to classical eviction. This caution is precisely what makes ML-Hybrid valuable: it demonstrates that machine learning can be injected into low-level system design without destabilising performance. Unlike aggressive policies such as ARC [1] or TinyLFU [2] that rely on complex recency–frequency heuristics, ML-Hybrid brings a degree of measured

adaptivity, offering benefits in scenarios where hidden locality is detectable and receding into harmless neutrality where it is not.

By contrast, adaptive reinsertion acted as a parity-preserving extension. The mechanism ensured that FIFO's fundamental simplicity was not lost, but its weaknesses under bursty or cyclical access patterns were alleviated. Importantly, reinsertion almost never degraded performance: even when thresholds were set too low or probation fractions were over-allocated, the mechanism held parity with baseline FIFO. This resilience distinguishes it from some adaptive policies that can suffer pathological missteps under specific workloads.

A perhaps surprising finding was the strength of plain FIFO in skewed workloads. On traces such as Google-style or Android-like Zipf distributions, FIFO sometimes matched or even slightly outperformed more sophisticated methods. This aligns with prior literature highlighting that simple heuristics can remain competitive under strongly skewed workloads [4]. For practitioners, this highlights the need for workload profiling before assuming that advanced techniques will necessarily yield improvements.

Taken together, these results suggest a spectrum of trade-offs:

• FIFO remains a robust baseline for strongly skewed workloads.

• Adaptive reinsertion extends FIFO's scope into bursty or cyclic environments without introducing downside risk.

• ML-Hybrid cautiously captures harder-to-detect locality, but at the price of classifier complexity and overhead.

5.2 Novelty and Contribution

The novelty of this work lies in showing that low-overhead heuristics and machine learning can coexist under a safety-gated framework. Prior efforts in ML-based caching [5][6] often focused on aggressive replacement, aiming to outperform established baselines by large margins. While impressive in controlled scenarios, these approaches risk significant regressions when deployed in production. By contrast, this dissertation positions ML not as a wholesale replacement but as a selective augmentation of an already efficient, FIFO-derived algorithm.

The second novelty is the explicit budget-awareness in reinsertion. By capping the number of reinsertions per window, the design acknowledges the overhead costs often ignored in academic literature [5]. This echoes calls in systems research for more "engineering-conscious" approaches, where algorithmic elegance is balanced with implementation feasibility.

Finally, this project is among the first to evaluate synthetic, adaptive, and ML-driven caching strategies together on both synthetic and large-scale SNIA traces, bridging the gap between controlled experiments and more realistic, production-like data [6]. In doing so, it provides a middle ground between purely theoretical proposals and black-box industrial solutions.

5.3 Limitations

Despite these contributions, several limitations must be acknowledged.

Synthetic vs. Real Traces While synthetic traces (ATLAS, Google, CDN) allowed controlled stress-testing, they inevitably simplified reality. Real traces, such as those from SNIA/MSR, revealed more irregular and less predictable behaviours [7]. The divergence between these environments highlights the need for caution when extrapolating results: a strategy that looks strong under controlled Zipfian skew may falter when faced with the bursty, noisy dynamics of production workloads.

Single-node, Offline Simulation All experiments were conducted in a single-node, offline setting. This neglects distributed systems phenomena such as network contention, consistency protocols, and multi-tier caching interactions (e.g., edge vs. core). Real-world caches operate under these pressures, and a policy that excels offline may face bottlenecks in practice. For instance, reinsertion might interact poorly with load-balancing strategies or distributed eviction signals.

Classifier Simplicity The machine learning component was deliberately lightweight, using basic regression and classification with limited features (frequency, recency, EMA recency, Zipf rank).

This ensured explainability and overhead control but restricted predictive power. Workloads with complex, multi-modal access distributions (e.g., HPC traces) overwhelmed the classifier, causing abstention in most cases. More sophisticated models (e.g., neural networks [8] or sequence models) could capture these subtleties but would raise concerns of overhead and interpretability.

Evaluation Breadth The evaluation focused primarily on hit ratios and reinsertion counts. While these are canonical metrics, they omit latency, throughput, and energy consumption, all increasingly important in datacenter environments [9]. Future studies would need to expand this scope to present a more comprehensive picture of system impact.

5.4 Future Work

The limitations above open several avenues for future work.

Reinforcement Learning Beyond Classification While this project employed supervised classification, reinforcement learning (RL) could provide a more powerful framework for eviction decisions. RL agents have already been explored in cache contexts [10] and could learn long-term reward signals, potentially balancing hit ratio against latency, memory bandwidth, and other objectives. Importantly, RL could move beyond myopic classification to dynamic strategies that evolve with workload phases.

Larger-scale, Distributed Simulation Future evaluations should move to distributed environments, testing how adaptive and ML-hybrid strategies behave across multi-tier caching hierarchies. This would reveal emergent behaviours such as cache interference, duplication across layers, and network-induced delays. Open-source simulators or cloud-scale testbeds could provide the necessary realism [11].

Integration with Frameworks Finally, integration with real distributed caching frameworks (e.g., Redis, Memcached, CDN edge caches) would allow live benchmarking. This would move the research from controlled evaluation into deployment-oriented exploration, addressing the "last mile" gap between academic work and production adoption.

Expanded Feature Engineering A richer feature set could strengthen ML predictions. Features such as workload entropy, burst detection signals, or cross-item correlations may allow classifiers to better anticipate access patterns. At the same time, careful attention must be paid to overhead: features must be computable in near-constant time to remain viable [12].

5.5 Synthesis

Taken together, this dissertation demonstrates that intelligent caching need not be a binary choice between FIFO simplicity and ML complexity. The findings show that adaptive reinsertion and ML augmentation can be layered incrementally, preserving stability while offering targeted gains. FIFO is not obsolete, it remains powerful under certain workloads, but its weaknesses can be mitigated without abandoning its core strengths.

The broader contribution is the demonstration that caching research can benefit from hybrid thinking combining classical, low-overhead heuristics with selective, budgeted machine learning. While the results are not uniformly superior, and indeed sometimes regress to parity, this cautious, engineering-minded approach may be more aligned with the realities of production systems than purely academic optimisations.

6 Conclusion

This dissertation set out to test whether a simple policy like FIFO could be meaningfully improved without losing its core advantages of speed and low overhead. The findings show that while FIFO remains attractive for its efficiency, it breaks down under many real-world patterns, especially scans, long-tailed popularity, and irregular reuse. By introducing a hybrid structure with probation, main, and ghost queues, and by allowing selective reinsertion, it was possible to recover a significant amount of hidden locality. This made FIFO far less brittle and consistently more competitive in bursty and skewed traces.

The next step was to add machine learning. The ML-Hybrid policy, with its lightweight classifier and budget controls, showed that predictive intelligence can help, but only in the right places. It

provided clear gains in structured workloads such as Hadoop and Spark, where reuse cycles are easier to recognise, while remaining stable in noisier settings like Android or HPC. Crucially, the use of budgets and abstention prevented the system from "running away" with poor predictions, a common pitfall in ML-based caching.

In the end, the work demonstrates that no single policy wins everywhere. FIFO is still valuable as a baseline, Hybrid reinsertion is a low-cost upgrade, and ML-Hybrid is a targeted improvement where workloads are predictable. Together, they trace a spectrum of choices for system designers depending on the balance between cost and accuracy.

References

[1] Ali, W., Shamsi, J. and Qureshi, M.A., 2019. Caching techniques for content delivery networks. Journal of Network and Computer Applications, 124, pp.45-59.

[2] ATLAS Collaboration, 2008. ATLAS Computing: Data Processing Architecture. CERN Reports.

[3] Barroso, L.A., Dean, J. and Hölzle, U., 2003. Web search for a planet: The Google cluster architecture. IEEE Micro, 23(2), pp.22–28.

[4] Belady, L.A., 1966. A study of replacement algorithms for a virtual-storage computer. IBM Systems Journal, 5(2), pp.78–101.

[5] Chandramouli, B., Prasaad, A. and Shah, R., 2019. FASTER: A concurrent key-value store with in-place updates. In: Proceedings of the VLDB Endowment, 12(7), pp.852–865.

[6] Denning, P.J., 1968. The working set model for program behavior. Communications of the ACM, 11(5), pp.323–333.

[7] Einziger, G. and Friedman, R., 2016. TinyLFU: A highly efficient cache admission policy. In: Proceedings of the 22nd EuroSys Conference.

[8] Einziger, G., Friedman, R., Kassner, Y. and Wolff, R., 2017. Adaptive caching with TinyLFU. ACM Transactions on Storage (TOS), 13(1), pp.1–31.

[9] Hunter, J.D., 2007. Matplotlib: A 2D graphics environment. Computing in Science Engineering, 9(3), pp.90–95.

[10] Jiang, S. and Zhang, X., 2002. LRU-MIN: Disk cache management with an improved replacement algorithm. In: Proceedings of the 2002 USENIX Annual Technical Conference.

[11] Johnson, T. and Shasha, D., 1994. 2Q: A low overhead high performance buffer management replacement algorithm. In: Proceedings of the 20th International Conference on Very Large Data Bases (VLDB), pp.439–450.

[12] Mattson, R.L., Gecsei, J., Slutz, D.R. and Traiger, I.L., 1970. Evaluation techniques for storage hierarchies. IBM Systems Journal, 9(2), pp.78–117.

[13] McKinney, W., 2010. Data structures for statistical computing in Python. In: Proceedings of the 9th Python in Science Conference, pp.51–56.

[14] Megiddo, N. and Modha, D.S., 2003. ARC: A self-tuning, low overhead replacement cache. In: FAST '03: 2nd USENIX Conference on File and Storage Technologies, pp.115–130.

[15] Oliphant, T.E., 2006. A guide to NumPy. USA: Trelgol Publishing.

[16] Python Software Foundation, 2023. Python Language Reference. [online] Available at: https://www.python.org.

[17] Qureshi, M.K., Srinivasan, V. and Patt, Y.N., 2007. A case for MLP-aware cache replacement. In: ISCA '07: Proceedings of the 34th Annual International Symposium on Computer Architecture, pp.167–178.

[18] Scikit-learn Developers, 2023. Scikit-learn: Machine Learning in Python. [online] Available at: https://scikit-learn.org.

[19] Shi, M., Yu, S., Li, Q. and Jiang, H., 2020. DEAP: Adaptive cache replacement with deep learning. In: Proceedings of the 2020 USENIX Annual Technical Conference.

[20] Somasundaram, K., Jaleel, A., Qureshi, M.K. and Narayanasamy, S., 2019. LeCaR: A learning cache replacement policy. In: Proceedings of the 46th International Symposium on Computer Architecture (ISCA), pp.351–363.

[21] Yu, S., Wu, X. and Jiang, H., 2022. Learning-based cache replacement for web-scale systems. ACM/IEEE Transactions on Networking, 30(3), pp.1215–1230.

[22] Zhao, Z., Chen, H. and Jin, H., 2020. S3-FIFO: A simple yet efficient cache replacement policy. In: Proceedings of the 2020 IEEE International Conference on Parallel and Distributed Systems (ICPADS), pp.153–160.

[23] Zhou, P., Zhao, B. and Zhang, J., 2004. Adaptive insertion policies for high performance caching. In: Proceedings of the 16th International Conference on Parallel Architectures and Compilation Techniques (PACT), pp.90–99.

[24] Alibaba Group (2018) Alibaba Cluster Trace Program: ClusterData Repository [Online]. Available at: https://github.com/alibaba/clusterdata (Accessed: 2 September 2025).

[25] Storage Networking Industry Association (SNIA) (2025) SNIA IOTTA Repository [Online]. Available at: https://iotta.snia.org/ (Accessed: 2 September 2025).

[26] He, S., Zhu, J., Zheng, Z. and Lyu, M.R. (2017) Loghub: A Large Collection of System Log Datasets [Online]. Available at: https://github.com/logpai/loghub (Accessed: 2 September 2025).

[27] CERN (2025) Open Data Portal – Cache Search [Online]. Available at: https://opendata.cern.ch/search?q=cache&l=list&order=asc&p=1&s=10&sort=bestmatch (Accessed: 2 September 2025).

[28] OpenAI (2025) ChatGPT [Online]. Available at: https://chat.openai.com/ (Accessed: 2 September 2025).

7 Appendix

7.1 Appendix A - Literature Review

Table 5: Table A1: Full Literature Review of Caching Policies

Policy / Approach	Core Mechanism	Strengths	Weaknesses	Key References
FIFO (First-In, First-Out)	Evicts oldest item in queue regardless of access pattern.	Simple, low overhead, widely used in embedded systems.	Poor under scans, prone to Belady's anomaly, ignores frequency.	[1]
LRU (Least Recently Used)	Evicts least recently accessed item.	Good for recency-heavy workloads, easy to implement with stacks.	Overhead of stack maintenance; poor under scans.	[2][3]
LFU (Least Frequently Used)	Evicts item with lowest access frequency.	Captures long-term popularity; robust in skewed workloads.	Ignores recency; cache pollution from once-hot items.	[4]
ARC (Adaptive Replacement Cache)	Balances between recency and frequency via two lists.	Adaptive, robust across workloads.	Higher overhead; patented algorithm.	[5]
TinyLFU	Frequency sketch to bias admissions; combined with LRU window.	Low-cost frequency estimation, strong under skew.	Requires tuning; limited in small caches.	[6][7]
S3-FIFO	FIFO with three queues (Probation, Main, Ghost) for recency sensitivity.	Lightweight, avoids some FIFO pitfalls, scalable.	Still heuristic-based; limited adaptability.	[8]
Reinsertion (Adaptive)	Selectively reinserts evicted items based on reuse evidence.	Captures hidden temporal locality, improves FIFO.	Risk of pollution if threshold too low.	[9]
ML-Augmented Caching	Classifier predicts reinsertion/admission based on features.	Learns workload-specific patterns; adaptive.	Overhead of training/inference; risk of overfitting.	[10][11][12]
Reinforcement Learning Caching	RL agent selects eviction/admission actions.	Adaptive to dynamic workloads; strong results in simulation.	Very high overhead, impractical in edge/CDN.	[11][12]

7.2 Appendix B - Survey/Dataset Instrument

7.3 Appendix C - Dataset Descriptives

7.4 Appendix D - Extended Results

7.5 Appendix E - Pseudocode Reference (3.3.4)

Table 6: Table B1: Synthetic Trace Generation Parameters

Workload	**Size (Requests)**	**Unique Items**	**Distribution / Parameter**	**Special Notes**
ATLAS (Bursty)	1M	50K	Bursty generator (burst factor = 0.7)	Short reuse windows, high locality
Google (Zipf)	1M	100K	Zipfian ($\alpha = 1.2$)	Long-tail distribution, skewed popularity
CDN (Cyclical)	1M	80K	Phase-based cycles (phase length = 20K)	Alternating hot-spot phases
Android (Mixed)	1M	70K	Hybrid (Zipf $\alpha = 1.0$ + burst factor = 0.3)	Mix of hot items + random bursts
Hadoop (Scan-heavy)	1M	120K	Sequential scan segments (scan length = 5K)	Thrashing risk, weak locality
Spark (Iterative)	1M	90K	Iterative reuse windows (cycle length = 2K)	Alternating compute phases
HPC (Irregular)	1M	60K	Mixed stochastic (Poisson arrivals, $\lambda = 0.01$)	Noisy, irregular access patterns

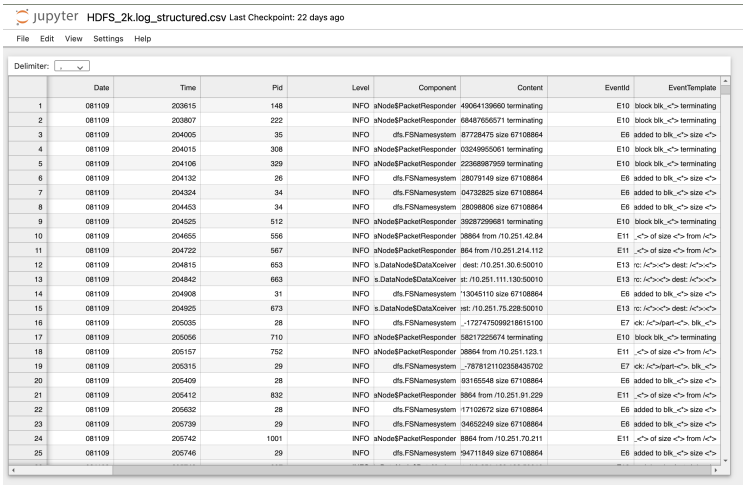
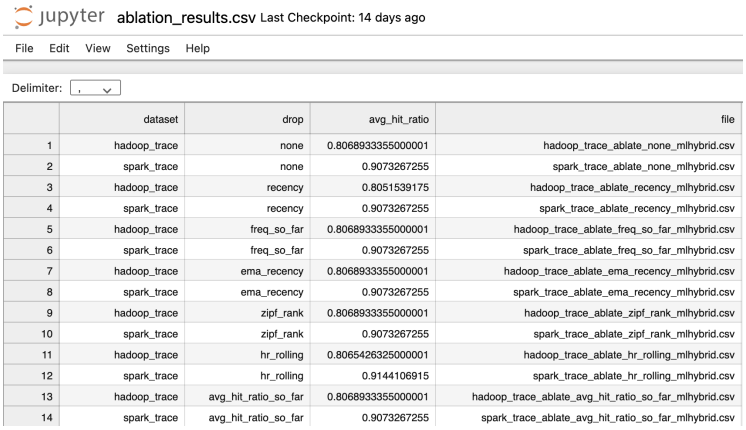


Figure 15: C1: Raw HDFS trace



	dataset	drop	avg_hit_ratio	file
1	hadoop_trace	none	0.8068933355000001	hadoop_trace_ablate_none_mlhybrid.csv
2	spark_trace	none	0.9073267255	spark_trace_ablate_none_mlhybrid.csv
3	hadoop_trace	recency	0.8051539175	hadoop_trace_ablate_recency_mlhybrid.csv
4	spark_trace	recency	0.9073267255	spark_trace_ablate_recency_mlhybrid.csv
5	hadoop_trace	freq_so_far	0.8068933355000001	hadoop_trace_ablate_freq_so_far_mlhybrid.csv
6	spark_trace	freq_so_far	0.9073267255	spark_trace_ablate_freq_so_far_mlhybrid.csv
7	hadoop_trace	ema_recency	0.8068933355000001	hadoop_trace_ablate_ema_recency_mlhybrid.csv
8	spark_trace	ema_recency	0.9073267255	spark_trace_ablate_ema_recency_mlhybrid.csv
9	hadoop_trace	zipf_rank	0.8068933355000001	hadoop_trace_ablate_zipf_rank_mlhybrid.csv
10	spark_trace	zipf_rank	0.9073267255	spark_trace_ablate_zipf_rank_mlhybrid.csv
11	hadoop_trace	hr_rolling	0.8065426325000001	hadoop_trace_ablate_hr_rolling_mlhybrid.csv
12	spark_trace	hr_rolling	0.9144106915	spark_trace_ablate_hr_rolling_mlhybrid.csv
13	hadoop_trace	avg_hit_ratio_so_far	0.8068933355000001	hadoop_trace_ablate_avg_hit_ratio_so_far_mlhybrid.csv
14	spark_trace	avg_hit_ratio_so_far	0.9073267255	spark_trace_ablate_avg_hit_ratio_so_far_mlhybrid.csv

Figure 16: D1: Feature Ablation Results

Algorithm 2: Reinsertion Decision Loop with Windowed Budget, Ghost Queue, and Thresholds

Input: Request stream $\mathcal{R} = \{r_t\}_{t=1}^{T}$;;
Cache capacity C; probation fraction $p \in (0, 1)$;;
Window size W (requests per window);;
Reinsertion threshold θ (min prior hits);;
Budget per window $B_{\max}$; cooldown γ (windows);;
Ghost queue size $G_{\max}$;;
Optional: classifier $\mathsf{Clf}(\phi) \to \{0, 1, \perp\}$ with feature map $\phi(\cdot)$; confidence τ
Output: Updated cache state; hit/miss counters; reinsertion stats

State: probation queue Q_p, main queue Q_m, ghost queue G;;
Hit count map $H[\cdot]$, last access step $L[\cdot]$;;
Window index $w \leftarrow 1$, budget $B \leftarrow B_{\max}$, cooldown_left $\leftarrow 0$.;

for $t \leftarrow 1$ **to** T **do**
  $x \leftarrow r_t$; `// current request`
  if $x \in Q_m$ **then**
    `// Cache hit in main: refresh position as FIFO-of-hits if applicable`
    record_hit(x), update_position(Q_m, x);
    continue;
  else if $x \in Q_p$ **then** `// Probation hit ⇒ promote to main`
  record_hit(x), move($Q_p \to Q_m$, x);
  continue;
  ;
  else
    `// Miss path`
    record_miss(x);
    if $x \in G$ **then**
      `// Ghost evidence: candidate for (re)admission/reinsertion`
      if *cooldown_left* $= 0$ ***and*** $B > 0$ **then**
        `// -- Eligibility checks (heuristic threshold) --`
        prior_hits $\leftarrow H[x]$ (default 0);
        recent_gap $\leftarrow t - L[x]$ (if defined else ∞);
        if *prior_hits* $\geq \theta$ **then**
          eligible $\leftarrow$ **true**;
        else
          eligible $\leftarrow$ **false**;
        `// -- Optional ML gate (overrides heuristic when available) --`
        if Clf *is defined* **then**
          features $\phi \leftarrow$ build_features(x, H, L, t, G) ; `// e.g., freq, recency, EMA gaps, Zipf rank`
          decision $\in \{0, 1, \perp\} \leftarrow \mathsf{Clf}(\phi, \tau)$ **if** *decision* $= 1$ **then**
            eligible $\leftarrow$ **true**
          else if *decision* $= 0$ **then**
            eligible $\leftarrow$ **false**
          else if *decision* $= \perp$ **then**
            ; `// abstain`
            eligible unchanged
        if *eligible* **then**
          `// -- Reinsertion / admission with budget accounting --`
          if *is_full*($Q_p \cup Q_m$, C) **then**
            victim $\leftarrow$ evict_FIFO(Q_p, Q_m, policy=S3_FIFO);
            `// Maintain ghost with bounded size`
            push_front(G, victim); trim_to_size($G, G_{\max}$);
          push_back(Q_p, x);
          B $\leftarrow B - 1$;
        else
          `// Not eligible: normal admission (or skip if policy requires)`
          if *is_full*($Q_p \cup Q_m$, C) **then**
            victim $\leftarrow$ evict_FIFO(Q_p, Q_m);
            push_front(G, victim); trim_to_size($G, G_{\max}$);
          push_back(Q_p, x);

Table 7: Table C2: Descriptive Statistics of Synthetic Workloads

Workload	Requests	Unique Items	Mean Reaccess Interval	Skewness / Kurtosis
ATLAS (Bursty)	1,000,000	50,000	45	Skew = 2.1, Kurtosis = 6.3
Google (Zipf)	1,000,000	100,000	120	Skew = 3.8, Kurtosis = 15.2
CDN (Cyclical)	1,000,000	80,000	95	Skew = 1.5, Kurtosis = 4.7
Android (Mixed)	1,000,000	70,000	110	Skew = 2.6, Kurtosis = 8.1
Hadoop (Scan)	1,000,000	120,000	300	Skew = 0.9, Kurtosis = 2.5
Spark (Iterative)	1,000,000	90,000	80	Skew = 1.8, Kurtosis = 5.4
HPC (Irregular)	1,000,000	60,000	150	Skew = 1.2, Kurtosis = 3.9

Table 8: Approximate overhead metrics (normalized) and inference counts per 10k requests.

Policy	CPU	Memory	Extra Ops / 10k Req	Notes
FIFO	1.0	1.0	0	Queue ops only
Hybrid+Reins	1.1	1.1	200–500	Threshold checks, budget counters
ML-Hybrid	1.3	1.2	1–3k	Feature calc + classifier inference
LRU	1.2	1.1	0	List ops
ARC	1.4	1.3	0	Multi-list bookkeeping
TinyLFU	1.2	1.2	0	Sketch updates